

คู่มือนักศึกษา

จากความต้องการสู่การสร้างระบบที่สอบย้อนกลับได้ (From Requirements to Traceable System)

Table of Contents

เกี่ยวกับคู่มือเล่มนี้	3
บทที่ 1 — ภาพรวมกระบวนการ: จาก “ตรวจผลงาน” สู่ “ตรวจความคิด”	4
แนวคิดเบื้องหลัง: QFD และ Functional Decomposition	4
ภาพรวม: 11 ก้าวการเรียนรู้ และ 6 จุดตรวจ.....	5
เส้นเลือดใหญ่ที่ร้อยทุกอย่างเข้าด้วยกัน: การสืบย้อน (Traceability).....	6
บทที่ 2 — วิธีใช้คู่มือนี้	7
บทที่ 3 — Requirements Foundation.....	8
แนวคิดเบื้องหลัง	8
ตัวอย่าง EEG SleepInsight.....	8
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	10
บทที่ 4 — House of Quality และเป้าหมายที่วัดได้	13
แนวคิดเบื้องหลัง	13
ตัวอย่าง EEG SleepInsight.....	14
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	17
บทที่ 5 — จาก QFD สู่ Functional Decomposition: ออกแบบระบบ	20
แนวคิดเบื้องหลัง	20
ตัวอย่าง EEG SleepInsight.....	21
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	22
บทที่ 6 — ลงมือสร้างและทดลอง	25
แนวคิดเบื้องหลัง	25
ตัวอย่าง EEG SleepInsight.....	25
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	26
บทที่ 7 — พิสูจน์และปิดการสืบย้อน	29
แนวคิดเบื้องหลัง	29

ตัวอย่าง EEG SleepInsight.....	30
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	31
บทที่ 8 — นำเสนอและปิดวงจรคุณค่า	33
แนวคิดเบื้องต้น	33
ตัวอย่าง EEG SleepInsight.....	33
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	34
นำนักคะแนนรวมทั้งกระบวนการ	36
บทสุดท้าย — รากทดแทนเมื่อไม่มีข้อมูลผู้ใช้จริง	37
ทำไมต้องมีบทนี้.....	37
แนวคิดเบื้องต้น	37
เดินตามตัวอย่าง: Intelligent Travel Planning AI Chatbot.....	37
สิ่งที่ต้องส่งเมื่อจบขั้นนี้	38
ภาคผนวก — ตัวอย่างเพิ่มเติมจากโครงการรูปแบบอื่น	41
ตัวอย่างที่ 1: API Gateway for Large Language Model Services (ซอฟต์แวร์ล้วน)	41
ตัวอย่างที่ 2: AIoT-based LPG Monitoring System (IoT).....	42

คู่มือนักศึกษา — กระบวนการ QFD → Functional Decomposition สำหรับวิชาโครงงานวิศวกรรมคอมพิวเตอร์ (Senior Project)

เกี่ยวกับคู่มือเล่มนี้

เอกสารฉบับนี้เป็นคู่มือสำหรับนักศึกษา อธิบายวิธีคิดและวิธีทำงานตลอดกระบวนการ ตั้งแต่ฟังเสียงผู้ใช้จนถึงส่งมอบและนำเสนอผลงาน แต่ละบทเดินเรื่องผ่านตัวอย่างจริงให้เห็นว่าทีมงานคิดอะไร ตัดสินใจอย่างไร และควรผลิตอะไรออกมา
ท้ายแต่ละบทมีสรุปเกณฑ์การให้คะแนนและแบบตรวจสอบตนเองให้ใช้ก่อนส่งงาน

คู่มือนี้จะรองรับ 2 รูปแบบ:

- เริ่มต้นจากศูนย์ (สำหรับโครงการที่ต้องการเริ่มต้นที่เก็บความต้องการผู้ใช้จริง):ให้อ่านทุกบทตามลำดับ
- ได้หัวข้อและมีความต้องการแล้ว จะไม่เก็บความต้องการผู้ใช้ใหม่: อ่านบทที่ 1-2 ก่อน แล้วข้ามไปอ่านบทสุดท้าย (“การทดแทนเมื่อไม่มีข้อมูลผู้ใช้จริง”) ก่อนย้อนกลับมาที่บทที่ 3 เป็นต้นไป

ตัวอย่างหลักที่ใช้ตลอดเล่ม: EEG SleepInsight — ระบบ AI ที่จำแนกระยะการนอนจากสัญญาณคลื่นสมอง
เดินครบทุกขั้นตอนตั้งแต่ต้นจนจบ ส่วนภาคผนวกท้ายเล่มมีตัวอย่างย่อจากอีก 2 โครงการ (ซอฟต์แวร์ล้วนและ IoT) ให้เห็นว่าวิธีคิดเดียวกันปรับใช้กับโครงงานรูปแบบอื่นได้อย่างไร

บทที่ 1 — ภาพรวมกระบวนการ: จาก “ตรวจผลงาน” สู่ “ตรวจความคิด”

ลองนึกภาพสถานการณ์นี้: เมื่อกรรมการสอบถามว่า “ทำไม accuracy เป้าหมายของโมเดลถึงต้องเป็น 80% ทำไมไม่ใช่ 70% หรือ 90%” นักศึกษาอาจตอบว่า “ก็ดูจากงานวิจัยที่คล้าย ๆ กันครับ” — ฟังดูสมเหตุสมผล แต่ถ้าถามต่อว่า “แล้วทำไมโครงการนี้ต้องแม่นยำเท่ากับงานวิจัยนั้น ผู้ใช้ของคุณต้องการแบบนั้นจริงหรือ” หลายครั้งคำตอบจะเงียบไป เพราะตัวเลขเป้าหมายนั้นไม่เคยถูกผูกกับความต้องการของผู้ใช้จริงตั้งแต่แรก มันแค่ “ดูสมเหตุผล” แต่ “สืบย้อนไม่ได้”

คู่มือนี้แก้ปัญหานี้ โดยเปลี่ยนสิ่งที่วัดผลจากผลงานสุดท้าย (สวยไหม ใช้งานง่ายไหม อธิบายเทคนิคได้ไหม) ไปเป็นกระบวนการคิด — ทุกตัวเลข ทุกฟังก์ชัน ทุกบรรทัดโค้ดที่เขียน ต้องสามารถสืบย้อนกลับไปหาความต้องการของผู้ใช้ที่แท้จริงได้เสมอ และในทางกลับกัน ทุกความต้องการของผู้ใช้ก็ต้องมีบางสิ่งที่พิสูจน์ได้ว่าโครงการทำสำเร็จจริง คุณสมบัตินี้เรียกว่า **traceability** (การสืบย้อนได้) และเป็นหัวใจของคู่มือทั้งเล่ม

แนวคิดเบื้องหลัง: QFD และ Functional Decomposition

QFD (Quality Function Deployment) เป็นวิธีคิดที่มาจากวิศวกรรมการผลิต ใช้แปลง “สิ่งที่ลูกค้าพูด” ให้กลายเป็น “สิ่งที่วิศวกรวัดและออกแบบได้” อย่างเป็นระบบ แทนที่จะให้วิศวกรเดาเอาเองว่าลูกค้าต้องการอะไร เครื่องมือหลักของ QFD เรียกว่า **House of Quality** (บ้านคุณภาพ) — เป็นตารางรูปทรงคล้ายบ้านที่จัดวางความต้องการของลูกค้าไว้ด้านหนึ่ง และคุณลักษณะทางเทคนิคที่วิศวกรควบคุมได้ไว้อีกด้านหนึ่ง แล้วให้ทีมช่วยกันตัดสินใจว่าอะไรตอบโจทย์อะไร

QFD ดั้งเดิมถูกออกแบบมาสำหรับสายการผลิตขนาดใหญ่ที่มีหลายทีมทำงานต่อกันเป็นทอด ๆ (จากแบบผลิตภัณฑ์ ไปถึงชิ้นส่วน ไปถึงกระบวนการผลิต ไปถึงสายพานจริง)

ซึ่งหนักเกินความจำเป็นสำหรับนักศึกษา 1-2 คนที่ทำโครงการซอฟต์แวร์ คู่มือนี้จึงตัดส่วนที่ไม่เกี่ยวข้อง (เช่น ขั้นตอนเกี่ยวกับชิ้นส่วนการผลิตและสายการผลิต) ออก เหลือไว้เฉพาะแก่นที่ยังใช้ได้จริงกับซอฟต์แวร์ คือการแปลงเสียงผู้ใช้เป็นคุณลักษณะที่วัดได้ในบ้านแห่งคุณภาพหนึ่งหลัง

เมื่อรู้แล้วว่าต้องควบคุมคุณลักษณะทางเทคนิคใดบ้าง คำถามต่อไปคือ

“แล้วระบบต้องทำอะไรบ้างถึงจะควบคุมคุณลักษณะนั้นได้” นี่คือหน้าที่ของ **Functional Decomposition** (การแตกฟังก์ชัน) — การแตกหน้าที่รวมของระบบออกเป็นหน้าที่ย่อยไปเรื่อย ๆ

จนถึงระดับที่จับคู่กับโมดูลของซอฟต์แวร์ได้จริง ชื่อของกระบวนการทั้งหมด “QFD ไปยัง Functional Decomposition” จึงหมายถึงการเดินทางสองช่วงต่อกัน: ช่วงแรกแปลงเสียงผู้ใช้เป็นตัวเลขเป้าหมาย (QFD) ช่วงที่สองแปลงตัวเลขเป้าหมายเป็นโครงสร้างซอฟต์แวร์ (Functional Decomposition)

ภาพรวม: 11 ก้าวการเรียนรู้ ย่อเหลือ 6 จุดตรวจ

กระบวนการทั้งหมดมี 11 ก้าวย่อยที่ควรทำความเข้าใจที่ละก้าว แต่ถ้าต้องส่งงานให้ตรวจ 11 ครั้ง
ทั้งนักศึกษาและอาจารย์ที่ปรึกษาจะเหนื่อยเกินไปโดยไม่จำเป็น คู่มือนี้จึงจัดกลุ่มก้าวย่อยเหล่านั้นให้เหลือ 6
จุดตรวจ (checkpoint) ที่มีความหมายในตัวเองชัดเจน แบ่งเป็น 3 ช่วง:

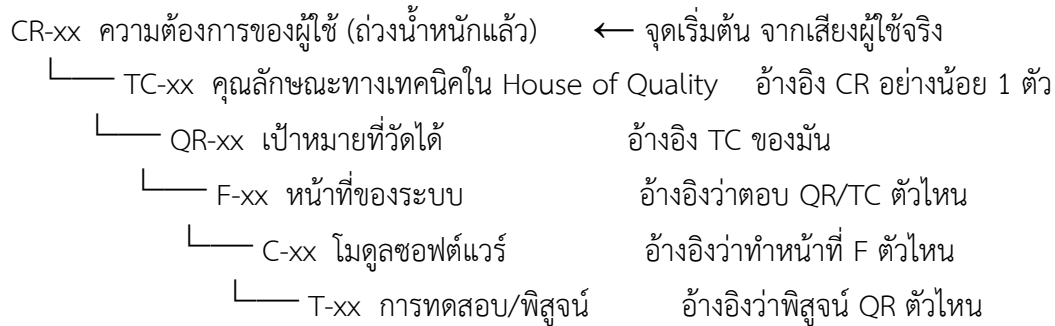
- **ช่วงฟังเสียงผู้ใช้และตั้งเป้า** — เก็บความต้องการจริงจากผู้ใช้ แปลงเป็นคุณลักษณะที่วัดได้ และตั้งตัวเลขเป้าหมาย (บทที่ 3-4)
- **ช่วงออกแบบ** — แยกเป็นหน้าที่ของระบบ แล้วจับหน้าที่เหล่านั้นลงในโมดูลซอฟต์แวร์ (บทที่ 5)
นี่คือขั้นที่ QFD เดินทางมาถึง Functional Decomposition อย่างเป็นรูปธรรม
- **ช่วงลงมือทำและพิสูจน์** — สร้างระบบจริง ทดสอบว่าทำได้ตามเป้าไหม
แล้วนำเสนอว่าตอบโจทย์ผู้ใช้จริงหรือเปล่า (บทที่ 6-8)

สามจุดตรวจแรกรวมกันเป็น proposal ฉบับสมบูรณ์ที่ส่งเมื่อจบช่วงวางแผน (ทำไปแล้วใน CEPP)
ส่วนสามจุดตรวจหลังกระจายอยู่ในช่วงลงมือสร้างและพิสูจน์ผลของวิชาโครงการ
ตารางด้านล่างสรุปโครงสร้างทั้งหมดไว้อ้างอิง

ขั้น	ชื่อ	ได้อะไร	รหัส
S0	ตั้งกรอบปัญหา + ผู้มีส่วนได้เสีย	Problem statement + stakeholder list	—
S1	เก็บเสียงผู้ใช้ (VoC)	VoC log ดิบ	—
S2	จัดกลุ่มเป็นความต้องการ	รายการ CR	CR
S3	ถ่วงน้ำหนักความสำคัญ	ตาราง CR ถ่วงน้ำหนัก	CR+weight
S4	House of Quality	บ้านคุณภาพ 1 ใบ	TC
S5	ตั้งเป้าที่วัดได้	สเปก QR	QR
S6	แตกฟังก์ชัน	Function tree	F
S7	จับลงสถาปัตยกรรม	แผนผังโมดูล	C
S8	ลงมือสร้าง + ทดลอง AI	ระบบทำงานได้	—
S9	ทดสอบ + ปิดการสืบทอด	ผล T + RTM ครบ	T
S10	นำเสนอผล	Demo + เทียบเป้า	—

เส้นเลือดใหญ่ที่ร้อยทุกอย่างเข้าด้วยกัน: การสืบย้อน (Traceability)

ลองนึกถึงใบเสร็จที่ดี — มันบอกได้ว่าสินค้าแต่ละชิ้นมาจากไหน ใครสั่ง ใครจ่าย ตรวจสอบย้อนกลับได้ทุกจุด
คู่มือนี้ใช้หลักการเดียวกันกับทุกชิ้นงานที่นักศึกษาผลิต โดยให้ทุกชิ้นมี “รหัส” ของตัวเอง
และทุกรหัสต้องบอกได้ว่ามันเกิดมาจากอะไร:



CR = Customer Requirement ความต้องการของผู้ใช้/ลูกค้า ที่ถ่วงน้ำหนักความสำคัญแล้ว

TC = Technical Characteristics หรือ Technical Requirements คุณลักษณะทางเทคนิคที่แปลงมาจาก CR

QR = Quality Rating หรือ Quality Requirement เป้าหมายที่วัดผ่าน/ไม่ผ่านได้จริง แปลงมาจาก TC

F = Functional ฟังก์ชันที่แตกจาก Functional Decomposition — ทุกโหนดต้องชี้ขึ้นไปหา QR/TC ที่มันตอบสนอง

C = Component / Module โมดูล/ส่วนประกอบของระบบที่ implement ฟังก์ชัน F นั้น

T = Test / Verification ทดสอบ/พิสูจน์ (Test, Demonstration, Inspection, Analysis) ที่ verify ว่า QR ผ่านจริง

ทำไมกฎ “ต้องอ้างอิงกันได้” ถึงสำคัญ? เพราะทุกครั้งที่มีจุดขาด มันมีความหมายเฉพาะเจาะจง: ถ้ามี CR ที่ไม่มี T ปลายทางเลย แปลว่าโครงการสัญญากับผู้ใช้อยู่แล้วแต่ไม่เคยพิสูจน์ว่าทำได้จริง — เป็น “คำสัญญาที่ไม่มีหลักฐาน”
ในทางกลับกัน ถ้ามี T ที่ไม่มี QR ต้นทาง แปลว่ากำลังทดสอบสิ่งที่ไม่มีใครต้องการ — เป็น
“งานที่ทำให้เกิดความจำเป็น” ง่ายขึ้น ๆ นี่เองที่ทำให้ตรวจสอบความคิดของตัวเองได้อย่างเป็นระบบ
แทนที่จะรู้สึกเอาเองว่า “น่าจะโอเค”

จุดที่มักสับสน: traceability ไม่ได้แปลว่าต้องมีเอกสารเยอะ ๆ หรือทำตารางให้ดูซับซ้อน —

มันแปลว่าทุกการตัดสินใจมีเหตุผลที่อธิบายได้และสืบย้อนได้ โครงการที่มี CR แค่ 5 ข้อแต่ทุกข้อสืบย้อนได้ชัด
ดีกว่าโครงการที่มี CR 20 ข้อแต่ครึ่งหนึ่งไม่รู้มาจากไหน

เดินตามตัวอย่างเดียวกันตลอดเล่ม

เพื่อให้เห็นภาพจริง คู่มือนี้ใช้โครงการสมมติชื่อ **EEG SleepInsight** — ระบบ AI

ที่จำแนกระยะการนอนจากสัญญาณคลื่นสมอง (EEG) — เดินตามทุกขั้นตอนตั้งแต่บทที่ 3 ถึงบทที่ 8

ให้เห็นว่าทีมงานคิดอะไร ตัดสินใจอย่างไร และรหัสแต่ละตัวโยงกันอย่างไรในทางปฏิบัติ

ส่วนภาคผนวกท้ายเล่มมีตัวอย่างย่อจากอีก 2 โครงการ (ซอฟต์แวร์ล้วนและ IoT)

ให้เห็นว่าหลักการเดียวกันนี้ใช้ได้กับโครงการหลากหลายรูปแบบ

บทที่ 2 — วิธีใช้คู่มือนี้

ลองนึกภาพว่านักศึกษาทำ House of Quality เสร็จแล้ว หลังจากนั้นจะมีคนสามฝ่ายมาดูงานชิ้นนี้: ตัวนักศึกษาเองก่อนจัดส่ง อาจารย์ที่ปรึกษาตอนตรวจงาน และเกณฑ์การให้คะแนนตอนประเมินผล ทั้งสามฝ่ายกำลังมองหาสิ่งเดียวกัน — งานชิ้นนี้สืบย้อนได้ครบไหม สมเหตุสมผลไหม — เพียงแค่มองจากมุมต่างกัน

นี่คือเหตุผลที่แต่ละจุดตรวจในคู่มือนี้ (บทที่ 3 ถึง 8) มีโครงสร้างเดียวกันเสมอ: อธิบายว่าขั้นนั้นสำคัญอย่างไร สอนแนวคิดเบื้องหลัง เดินตามตัวอย่างจริงที่ละก้าว บอกว่าต้องส่งอะไร

แล้วปิดท้ายด้วยเกณฑ์การให้คะแนนและแบบตรวจสอบตนเอง

เพราะทั้งหมดอ้างอิงรายการเกณฑ์ชุดเดียวกันในแต่ละชั้น เนื้อหาทุกส่วนจึงพูดเป็นเสียงเดียวกัน — ไม่มีทางที่ rubric จะเข้มกว่าที่บทเรียนสอนไว้ หรือ self-check จะถามคนละเรื่องกับที่จะถูกให้คะแนน

วิธีอ่านแต่ละบท: เนื้อหาส่วนต้นของบท (ทำไมสำคัญ + แนวคิด + ตัวอย่างที่เดินตามทีมงาน)

ควรอ่านก่อนลงมือทำ ส่วนท้ายบทมีกล่องสรุปสองกล่อง — เกณฑ์การให้คะแนน และแบบตรวจสอบตนเอง — แยกไว้ให้ชัดเจนด้วยเส้นคั่น เพื่อกลับมาเปิดใช้สะดวกตอนใกล้ส่งงาน โดยไม่ต้องไล่หาในเนื้อหา

เรื่องน้ำหนักระเมินรวมทั้งวิชา: แต่ละจุดตรวจ (checkpoint) ทั้ง 6

จุดมีน้ำหนักเท่ากันในการประเมินกระบวนการโดยรวม (checkpoint ละประมาณ 16.7%)

รายละเอียดอยู่ที่ท้ายบทที่ 8

บทที่ 3 — Requirements Foundation

ทุกสิ่งที่จะเกิดขึ้นตลอดโครงการ ตั้งแต่คุณลักษณะทางเทคนิค ไปจนถึงบรรทัดโค้ดสุดท้ายที่ผ่านการทดสอบ ล้วนต้องสืบย้อนกลับมาถึงจุดเริ่มต้นจุดเดียว: ความต้องการของผู้ใช้จริงที่เก็บมาได้ในขั้นนี้ ถ้าหากนี้ไม่แน่น ทุกอย่างที่สร้างต่อจากมันก็ล่อยไปด้วย นี่คือเหตุผลที่ขั้นนี้ชื่อว่า “Requirements Foundation” —

แนวคิดเบื้องต้น

Voice of Customer (VoC) คือคำพูดดิบของผู้ใช้ ก่อนที่ใครจะตีความ กฎสำคัญคืออย่ารีบตีความ — ถ้าผู้ใช้พูดว่า “อยากรู้ว่าคืนนี้หลับลึกพอไหม” ให้บันทึกไว้แบบนั้น อย่าเพิ่งแปลงเป็น “ต้องการ dashboard แสดงคุณภาพการนอน” เพราะการตีความเร็วเกินไปอาจพาไปผิดทาง คำพูดดิบต้องถูกเก็บไว้ก่อน แล้วค่อยตีความในขั้นถัดไปอย่างมีสติ

การเก็บ VoC ทำได้หลายวิธี — สัมภาษณ์ตัวต่อตัว สังเกตพฤติกรรมจริง หรือแบบสอบถามปลายเปิด สิ่งสำคัญไม่ใช่วิธีไหนดีที่สุด แต่คือต้องมาจากคนจริง ไม่ใช่การนั่งจินตนาการเอาเอง และควรเก็บจากหลายกลุ่มผู้มีส่วนได้เสีย ไม่ใช่แค่เพื่อนในทีมหรือคนกลุ่มเดียว เพื่อให้เห็นภาพที่ครบและไม่ลำเอียง

ขั้นถัดไปคือ **affinity grouping** (การจัดกลุ่มตามความใกล้เคียง) — เอาคำพูดดิบทั้งหมดมาดูว่าประโยคไหนพูดถึงเรื่องเดียวกัน แม้จะใช้คำต่างกัน แล้วรวมกลุ่มเป็นความต้องการเดียว วิธีนี้เรียกอีกชื่อว่า KJ method ตามชื่อนักมานุษยวิทยาชาวญี่ปุ่นผู้คิดค้น ผลลัพธ์ของขั้นนี้คือรายการ CR (Customer Requirement) — ข้อควรระวังที่สำคัญที่สุดคือ **CR ต้องเป็น “ความต้องการ” ไม่ใช่ “วิธีแก้”** ตัวอย่างเช่น “อยากเห็นผลไว” เป็นความต้องการ แต่ “ใช้ WebSocket” เป็นวิธีแก้ — ถ้าเปลี่ยนเทคโนโลยีไปใช้ออย่างอื่น ความต้องการ “อยากเห็นผลไว” ยังคงอยู่เสมอ นี่คือนิยามที่แตกต่างอะไรควรอยู่ในขั้นนี้ การรีบกระโดดไปคิดวิธีแก้ก่อนเข้าใจความต้องการให้ชัด เป็นกับดักที่พบบ่อยที่สุดของขั้นนี้ เพราะมันตัดโอกาสที่จะเห็นทางเลือกอื่นที่อาจดีกว่า

สุดท้ายคือการ **ถ่วงน้ำหนัก** — ไม่ใช่ทุกความต้องการสำคัญเท่ากัน วิธีง่ายที่สุดคือให้คะแนน 1–5 โดยตรง หรือแจกคะแนนรวม 100 คะแนนให้ทุก CR ช่วยกันแบ่ง ส่วนวิธีที่ละเอียดกว่า (เรียกว่า AHP หรือ Analytic Hierarchy Process) ใช้การเทียบคู่ทีละคู่แล้วคำนวณน้ำหนักจากผลเปรียบเทียบทั้งหมด ซึ่งให้ผลแม่นยำกว่าแต่ใช้เวลามากขึ้นตามจำนวนความต้องการที่เพิ่มขึ้น เหมาะกับกรณีที่มีลูกค้าจริงให้สัมภาษณ์เจาะลึก สำหรับโครงการส่วนใหญ่ วิธีถ่วงน้ำหนักตรง ๆ ก็เพียงพอ ขอเพียงระบุที่มาของน้ำหนักให้ชัดเจนว่าได้มาอย่างไร ไม่ใช่ตัวเลขที่คิดขึ้นเองลอย ๆ

ตัวอย่าง EEG SleepInsight

EEG SleepInsight เริ่มต้นด้วยคำถามพื้นฐานที่สุด: ใครคือคนที่จะได้ประโยชน์จากระบบนี้ และพวกเขากำลังเจอปัญหาอะไรอยู่ คำตอบที่ได้กลายเป็น problem statement:

“การประเมินคุณภาพการนอนที่แม่นยำต้องทำ polysomnography ในห้องแล็บ ซึ่งแพง ใช้เวลา และติดอิเล็กทรอนิกส์หลายจุด ทำให้คนทั่วไปเข้าถึงยาก จึงต้องการระบบ AI ที่จำแนกระยะการนอนจากสัญญาณ EEG จำนวน channel น้อยได้เชื่อถือพอ”

ทีมระบุผู้มีส่วนได้เสียไว้สามกลุ่ม: ผู้ใช้ทั่วไปที่อยากติดตามการนอนของตัวเอง แพทย์หรือนักเทคนิคการนอนที่อาจนำผลไปใช้ต่อ และนักวิจัยด้าน sleep/EEG สังเกตว่าไม่ใช่แค่ “ผู้ใช้” กลุ่มเดียว แต่ครอบคลุมทุกคนที่มีส่วนเกี่ยวข้องกับความสำเร็จของระบบ — เพราะถ้าลืมนักเทคนิคการนอนกลุ่มหนึ่งไป ความต้องการของกลุ่มนั้นก็จะไม่ถูกเก็บมาด้วย ทีมสรุปกลุ่มพร้อมบทบาทไว้เป็นตาราง เพื่อให้ตรวจสอบทีหลังได้ว่าเก็บเสียงมาจากทุกกลุ่มจริง:

กลุ่มผู้มีส่วนได้เสีย	บทบาทในระบบ	ทำไมต้องฟังเสียงกลุ่มนี้
ผู้ใช้ทั่วไป	ผู้ใช้งานหลัก ติดตามคุณภาพการนอนของตัวเองที่บ้าน	เป็นกลุ่มที่ระบบต้องตอบโต้โดยตรงทุกคืน
แพทย์ / นักเทคนิคการนอน	ผู้ใช้ผลลัพธ์ต่อในบริบททางการแพทย์ (คัดกรองเบื้องต้น)	กำหนดว่าความแม่นยำต้อง “น่าเชื่อถือ” ระดับไหนถึงเอาไปใช้ต่อได้
นักวิจัยด้าน sleep/EEG	ผู้ที่อาจนำระบบหรือข้อมูลไปต่อยอดงานวิจัย	สนใจความโปร่งใสของวิธีวัดและข้อจำกัดของโมเดล

ขั้นตอนมาคือการฟังเสียงจริง ๆ โดยไม่รีบตีความ ทีมสัมภาษณ์ผู้ใช้กลุ่มต่าง ๆ แล้วได้คำพูดดิบมาหกประโยค พร้อมระบุไว้ทุกครั้งว่าใครพูดและเก็บมาด้วยวิธีไหน เพื่อให้ VoC log ตรวจสอบที่มาย้อนกลับได้ ไม่ใช่แค่ก็อปคำพูดมาลอย ๆ:

รหัส	คำพูดของผู้ใช้	ผู้พูด (กลุ่ม)	วิธีเก็บ
V1	“อยากรู้ว่าคืนนี้หลับลึกพอไหม โดยไม่ต้องไปนอนที่ รพ.”	ผู้ใช้ทั่วไป (คนที่ 2)	สัมภาษณ์ตัวต่อตัว
V2	“อุปกรณ์ที่บ้านต้องใส่ง่าย ไม่ใช่ติดสาย 10 เส้น”	ผู้ใช้ทั่วไป (คนที่ 4)	สัมภาษณ์ตัวต่อตัว
V3	“ผลต้องอ่านรู้เรื่อง ไม่ใช่กราฟที่ต้องเป็นหมอลงใจ”	ผู้ใช้ทั่วไป (คนที่ 1)	แบบสอบถามปลายเปิด
V4	“ต้องแม่นยำพอที่หมอจะเอาไปดูต่อได้”	นักเทคนิคการนอน	สัมภาษณ์ตัวต่อตัว
V5	“อยากเห็นผลตอนเช้าเลย ไม่ต้องรอหลายวัน”	ผู้ใช้ทั่วไป (คนที่ 3)	แบบสอบถามปลายเปิด
V6	“เครื่องต้องทำงานได้ทั้งคืนไม่หลุด”	ผู้ใช้ทั่วไป (คนที่ 2)	สัมภาษณ์ตัวต่อตัว

สังเกตว่า V1 กับ V5 ใช้คำต่างกัน แต่พูดถึงเรื่องเดียวกันคือ “อยากเห็นผลเร็ว ไม่ต้องรอนาน” ทีมจึงจัดสองประโยคนี้อยู่กลุ่มเดียวกัน ส่วน V4 ซึ่งพูดถึงความน่าเชื่อถือระดับที่แพทย์ยอมรับได้ ถูกแยกไว้เป็นอีกกลุ่มหนึ่งเพราะเป็นคนละประเด็นกับความเร็ว แม้ทั้งคู่จะฟังดูเกี่ยวกับ “คุณภาพ” ก็ตาม นี่คือทักษะสำคัญของการจัดกลุ่ม — ต้องแยกแยะว่าอะไรคือธีมเดียวกันจริง ๆ ไม่ใช่แค่ใช้คำคล้ายกัน ทีมสรุปผลการจัดกลุ่ม (affinity grouping) เป็นตารางก่อนตั้งชื่อความต้องการ เพื่อให้เห็นว่าแต่ละธีมรวบรวมคำพูดกี่ประโยค:

จิม (affinity group)	VoC ที่รวมกัน	กลายเป็น
ความแม่นยำที่แพทย์ยอมรับได้	V4	CR-01
อุปกรณ์ใส่ง่าย channel น้อย	V2	CR-02
อยากเห็นผลเร็ว ไม่ต้องรอนาน	V1, V5	CR-03
ผลอ่านเข้าใจง่ายสำหรับคนทั่วไป	V3	CR-04
ทำงานเสถียรตลอดคืน	V6	CR-05

ผลลัพธ์จากการจัดกลุ่มทั้งหมดออกมาเป็นความต้องการ 5 ข้อ พร้อมระบุที่มาไว้ทุกข้อ เพื่อให้ตรวจสอบย้อนกลับได้ว่าความต้องการแต่ละข้อไม่ได้ถูกคิดขึ้นเอง:

รหัส	ความต้องการ	มาจาก	น้ำหนัก
CR-01	จำแนกระยะนอนแม่นยำเชื่อถือได้	V4	5
CR-02	อุปกรณ์ใส่ง่าย ใช้ channel น้อย	V2	4
CR-03	เห็นผลไว (เกือบทันที)	V1, V5	3
CR-04	ผลอ่านเข้าใจง่ายสำหรับคนทั่วไป	V3	3
CR-05	ทำงานเสถียรตลอดคืน	V6	4

น้ำหนักไม่ได้มาจากการเดา ทีมกลับไปถามผู้ใช้ตรง ๆ ว่าอะไรสำคัญที่สุด และได้คำตอบตรงกันว่าความแม่นยำ (CR-01) คือสิ่งที่ขาดไม่ได้เลย ตามด้วยความสะดวกในการใช้อุปกรณ์ ส่วนความเร็วในการเห็นผลและความเข้าใจง่าย แม้สำคัญ แต่ผู้ใช้ยอมรับได้หากต้องแลกกับความแม่นยำที่สูงขึ้น น้ำหนักเหล่านี้เองที่จะเป็นตัวกำหนดว่าคุณลักษณะทางเทคนิคตัวไหนสำคัญที่สุดในบทรัดไป

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- Problem statement + รายชื่อผู้มีส่วนได้เสียพร้อมบทบาท
- VoC log ดิบ (คำพูดผู้ใช้ + ระบุว่าใครพูด เก็บมาอย่างไร)
- แผนผังการจัดกลุ่ม (affinity/KJ diagram)
- ตาราง CR พร้อมน้ำหนักและที่มา

จำนวนที่แนะนำ (เป็นช่วงปกติ ไม่ใช่กฎตายตัว — ถ้าน้อยหรือมากกว่านี้ให้อธิบายเหตุผลไว้):
 ผู้มีส่วนได้เสียอย่างน้อย 3 กลุ่ม, คำพูดดิบ 10–30 ประโยค, **ความต้องการ (CR) 5–12 ข้อ**

ข้อผิดพลาดที่พบบ่อย: เขียน CR ที่ “คิดเอง” โดยไม่มี VoC รองรับ · เขียน CR เป็นวิธีแก้ (“ใช้ deep learning”) แทนความต้องการ · VoC เป็นการตีความของผู้ทำเองไม่ใช่คำผู้ใช้จริง ไม่ระบุวิธีหรือที่มาของน้ำหนัก

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่าน	2 ต้องแก้	1 ควรปรับปรุง
ทุก CR สืบย้อน VoC ได้ (25)	ชี้ได้ชัดทุกตัว	ลอย ≤ 1 ตัว	ลอย 2-3 ตัว	ส่วนใหญ่ไม่มี VoC รองรับ
CR ครอบคลุมธีมของ VoC (20)	ครบทุกธีม	ตกหล่นเล็กน้อย	ตกหล่นธีมสำคัญ	ไม่สะท้อน VoC
VoC มีคุณภาพและมีแหล่งที่มา (20)	VoC จริง หลากหลาย มีแหล่งที่มา	ส่วนใหญ่ดี	น้อย/กลุ่มเดียว	ไม่มี VoC จริง
CR เป็นความต้องการ ไม่ใช่วิธีแก้ (15)	เป็นความต้องการทั้งหมด	เอียงไปวิธีแก้ 1-2	หลายตัวเป็นวิธีแก้	เป็น feature list
น้ำหนักมีที่มาโปร่งใส (10)	ครบ + มีวิธี	ครบ วิธีคลุมเครือ	บางส่วน	ไม่มี
Problem statement + stakeholder ครบ (10)	ชัดเจน ครบ	ครบแต่สั้น	ไม่ครบ	ไม่ชัด

น้ำหนักรวม 100 คะแนน: การสืบย้อนได้กลับไปหาความต้องการจริงหนักที่สุดที่ 45%
 เพราะนี่คือรากของกระบวนการทั้งหมด

ตรวจสอบตนเองก่อนส่งงาน

- [] Problem statement มีบริบท + เหตุผลที่ควรแก้
- [] ผู้มีส่วนได้เสีย ≥ 3 กลุ่ม แต่ละกลุ่มระบุบทบาท
- [] VoC เป็นคำผู้ใช้จริง + ระบุแหล่งที่มา
- [] CR ทุกตัวชี้กลับ VoC ได้ (ไม่มี CR ที่คิดเอง)
- [] CR เป็น "ความต้องการ" ไม่ใช่วิธีแก้/เทคโนโลยี
- [] CR ทุกตัวมีน้ำหนัก + บอกวิธีถ่วงน้ำหนัก
- [] รีม VoC หลักกลายเป็น CR ครบ ไม่ตกหล่น/ซ้ำ

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจ "Requirements Foundation" (VoC $\rightarrow$ Customer Requirements)
ตรวจเข้มงวดตามเกณฑ์ รายงานเป็นตาราง PASS/FAIL + เหตุผล + จุดที่ต้องแก้ไข

ข้อมูลของฉัน:

- Problem statement: [วาง]
- Stakeholder list (กลุ่ม + บทบาท): [วาง]
- VoC log ดิบ (คำผู้ใช้ + แหล่งที่มา): [วาง]
- Affinity/CR mapping (CR + VoC ที่มา): [วาง]
- ตาราง CR + น้ำหนัก + วิธีถ่วงน้ำหนัก: [วาง]

ตรวจทีละข้อ:

1. VoC ประโยคไหนเป็น "การตีความของผู้ทำ" ไม่ใช่คำผู้ใช้จริง — ชี้ตัว
2. มี CR ตัวไหน "ลอย" คือชี้กลับ VoC item ไม่ได้ใหม่ — ชี้ตัว (เกณฑ์สำคัญสุด)
3. CR ตัวไหนเขียนเป็นวิธีแก้/เทคโนโลยี แทนความต้องการ — ชี้ตัว + เสนอวิธีเขียนใหม่
4. มีรีมใน VoC ที่ยังไม่กลายเป็น CR (ตกหล่น) ใหม่ / มี CR ซ้ำซ้อนไหม
5. CR ทุกตัวมีน้ำหนักและระบุวิธีถ่วงน้ำหนักไหม — ชี้ที่ขาด

สุดท้าย:

- วาด traceability 1 เส้นเต็ม: VoC "..." $\rightarrow$ CR-?? และชี้ CR ที่ไม่มีรายการรับ
- สรุป 3 อย่างที่ต้องแก้ก่อนเป็นอันดับแรก

อย่าแต่งข้อมูลเพิ่มเอง

บทที่ 4 — House of Quality และเป้าหมายที่วัดได้

เมื่อมีรายการความต้องการที่ถ่วงน้ำหนักแล้วจากบทก่อน คำถามที่ตามมาคือ

“แล้วเราจะรู้ได้อย่างไรว่าระบบตอบสนองต่อโจทย์ความต้องการเหล่านั้นจริง”

คำตอบคือต้องแปลงความต้องการที่เป็นคำพูดของผู้ใช้ ให้กลายเป็นสิ่งที่วัดเป็นตัวเลขได้ — นี่คือหน้าที่ของ House of Quality บทนี้จึงเป็นจุดเชื่อมต่อสำคัญระหว่าง “สิ่งที่ผู้ใช้ต้องการ” กับ “สิ่งที่วิศวกรออกแบบและทดสอบได้”

แนวคิดเบื้องหลัง

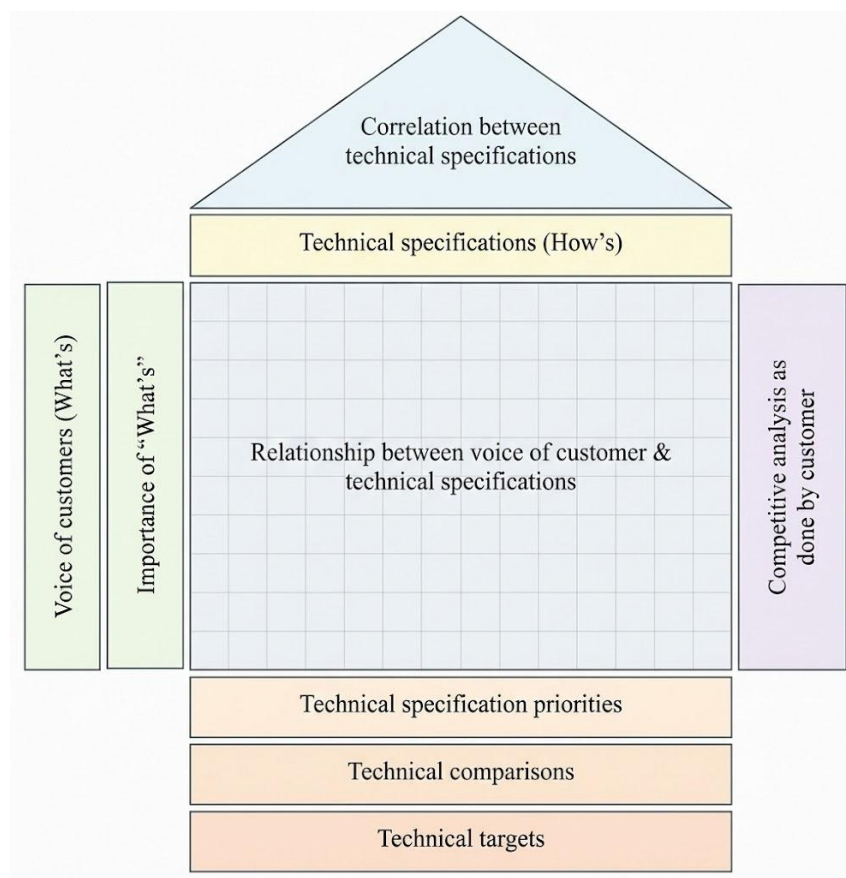
House of Quality เปรียบเสมือนบ้านที่มีหลายห้อง แต่ละห้องเก็บข้อมูลคนละชนิด:

ผนังซ้ายเก็บความต้องการของผู้ใช้ (CR) ที่ถ่วงน้ำหนักแล้วจากบทก่อน เพดานเก็บ**คุณลักษณะทางเทคนิค** (TC)

ที่เป็นตัวแปรวัดได้ซึ่งวิศวกรควบคุมได้จริง ตัวบ้านตรงกลางเป็นตารางความสัมพันธ์ระหว่างสองฝั่ง

หลังคาเก็บว่าคุณลักษณะทางเทคนิคแต่ละตัวส่งผลต่อกันอย่างไร

และฐานบ้านเก็บตัวเลขเป้าหมายของแต่ละคุณลักษณะ



ตัวอย่าง House of Quality : ด้านซ้ายจะเป็น VoC เพดานหรือด้านบนเก็บ TC ภายในบ้านเก็บความสัมพันธ์ระหว่าง 2 ฝั่งหลังคา และฐานจะเก็บตัวเลขเป้าหมายแต่ละ TC

จุดสำคัญที่สุดในการแปลง CR เป็น TC คือคุณลักษณะทางเทคนิคต้องวัดได้จริง ไม่ใช่คำคุณศัพท์ลอย ๆ เช่น “ใช้งานง่าย” หรือ “แม่นยำ” ต้องระบุมิติที่ชัดเจน เช่น “ความแม่นยำในการจำแนก” หรือ “จำนวน channel ที่ต้องใช้” เพื่อให้การเลือกค่าเป็นระบบและไม่ต้องคิดค่าใหม่ทุกครั้ง คู่มือนี้แนะนำให้ใช้คำศัพท์จากมาตรฐาน ISO/IEC 25010 ซึ่งเป็นมาตรฐานสากลที่จัดหมวดคุณลักษณะคุณภาพซอฟต์แวร์ไว้แล้ว เช่น functional suitability (ความถูกต้องของหน้าที่การทำงาน), performance efficiency (ประสิทธิภาพ), usability (ความง่ายในการใช้งาน), reliability (ความน่าเชื่อถือ) — การอิงมาตรฐานนี้ช่วยให้ไม่ต้องเริ่มคิดจากศูนย์ และทำให้ทีมอื่นเข้าใจสิ่งที่กำลังพูดถึงได้ทันที

เมื่อได้ TC แล้ว ขั้นตอนต่อไปคือให้คะแนนความสัมพันธ์ระหว่าง CR แต่ละตัวกับ TC แต่ละตัวในตารางกลางบ้าน (เรียกว่า relationship matrix) โดยใช้สเกลมาตรฐาน 9 = สัมพันธ์แรง, 3 = ปานกลาง, 1 = อ่อน, ว่าง = ไม่เกี่ยวข้อง ตัวเลขนี้ไม่ได้ให้แบบสุ่ม แต่ต้องผ่านการคิดจริงว่า TC ตัวนี้ช่วยตอบ CR ตัวนั้นมากแค่ไหน

ส่วนหลังคาของบ้านคือจุดที่มักถูกมองข้าม แต่สำคัญมากโดยเฉพาะกับโครงการที่มี AI เป็นแกนหลัก — มันบันทึกความขัดแย้งระหว่างคุณลักษณะทางเทคนิค เช่น ยิ่งโมเดลแม่นยำขึ้น มักยิ่งใช้เวลาประมวลผลนานขึ้น (ความแม่นยำกับความเร็วขัดกัน) หรือยิ่งใช้ channel สัญญาณมากขึ้น มักยิ่งแม่นยำขึ้นแต่ใช้งานยากขึ้น (ความแม่นยำกับความสะดวกขัดกัน) การเห็นความขัดแย้งเหล่านี้ล่วงหน้าช่วยให้ทีมตัดสินใจอย่างมีข้อมูล แทนที่จะไปเจอปัญหาตอนสร้างจริงแล้วต้องแก้ไขย้อนหลัง

สุดท้ายคือการคำนวณน้ำหนักความสำคัญของแต่ละ TC โดยคุณน้ำหนักของ CR กับระดับความสัมพันธ์ในตารางกลางบ้าน แล้วรวมกันทุก CR — TC ตัวที่ได้คะแนนสูงสุดคือตัวที่ควรให้ความสำคัญที่สุดในการออกแบบ จากนั้นจึงใส่ตัวเลขเป้าหมายเบื้องต้น (target value) ลงในฐานบ้าน

เมื่อรู้แล้วว่า TC ตัวไหนสำคัญที่สุด ขั้นสุดท้ายคือแปลง TC เหล่านั้นให้เป็น Quantifiable Requirement (QR) — ข้อกำหนดที่ระบุตัวเลขเป้าหมายชัดเจนจนทดสอบได้จริง ข้อกำหนดที่ดีต้องมีครบ 5 องค์ประกอบ: **metric** (ตัวชี้วัดคืออะไร), **operator** (มากกว่า น้อยกว่า หรือเท่ากับ), **threshold** (ตัวเลขเท่าไร), **dataset** (วัดบนข้อมูลชุดไหน), และ **condition** (ภายใต้เงื่อนไขอะไร) — ขาดองค์ประกอบใดไปหนึ่งอย่าง ข้อกำหนดนั้นจะทดสอบไม่ได้จริง ซึ่งขัดกับหลักการพื้นฐานของวิศวกรรมความต้องการที่ว่า “ข้อกำหนดที่ดีต้องพิสูจน์ได้ด้วยกระบวนการที่ชัดเจน”

ตัวอย่าง EEG SleepInsight

ทีมงานเริ่มต้นด้วยการไล่ทีละ CR แล้วถามตัวเองว่า

“จะวัดหรือควบคุมความต้องการข้อนี้ด้วยตัวแปรทางเทคนิคอะไร” สำหรับ CR-01 (จำแนกระยะนอนแม่นยำ) คำตอบตรงตัวคือความแม่นยำในการจำแนก ส่วน CR-02 (ใช้ channel น้อย) แปลงตรงเป็นจำนวน channel ที่ต้องใช้ ทำแบบนี้ไปทีละตัวจนได้ตารางแปลง:

CR (น้ำหนัก)	→ TC ที่ตอบ (ตามหมวด ISO 25010)
CR-01 จำแนกระยะนอนแม่นยำเชื่อถือได้ (5)	TC-01 ความแม่นยำการจำแนก 5 ระยะ (<i>functional correctness</i>)
CR-02 ใช้กับ EEG อุปกรณ์หาง่าย channel น้อย (4)	TC-02 จำนวน EEG channel ที่ต้องใช้ (<i>portability/compatibility</i>)
CR-03 เห็นผลไว ติดตามการนอนได้ (3)	TC-03 latency ต่อ epoch 30 วินาที (<i>performance efficiency</i>)
CR-04 ผลอ่านเข้าใจง่าย (ไม่ใช่แพทย์) (3)	TC-04 ความชัดของ hypnogram/visualization (<i>usability</i>)
CR-05 ทำงานเสถียรตลอดคืน (4)	TC-05 ความทนต่อ artifact/สัญญาณขาด (<i>reliability</i>)

	TC-01 ความแม่นยำการจำแนก 5 ระยะ	TC-02 จำนวน EEG channel	TC-03 latency ต่อ epoch	TC-04 ความชัดของ hypnogram	TC-05 ความทนต่อ artifact
CR-01 จำแนกระยะนอนแม่นยำเชื่อถือได้ (5)	9	3			
CR-02 ใช้ EEG channel น้อย (4)		9			
CR-03 เห็นผลไว (3)			9		
CR-04 ผลอ่านเข้าใจง่าย (3)					
CR-05 ทำงานเสถียรตลอดคืน (4)					9

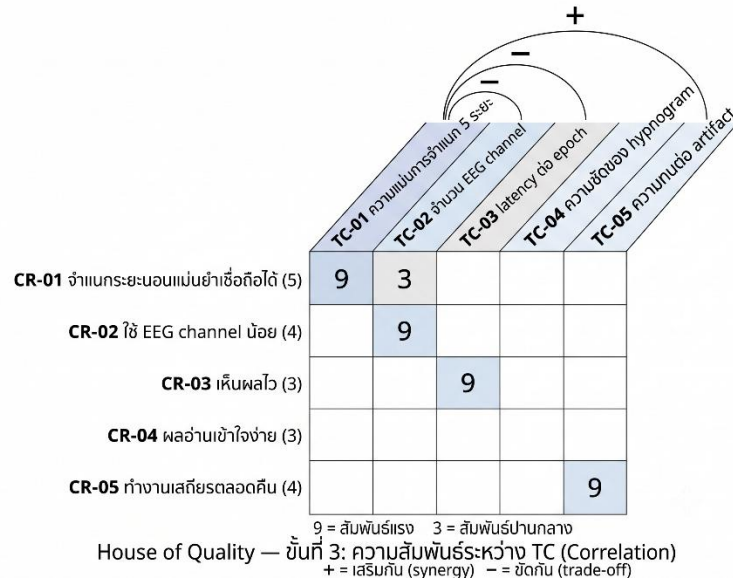
9 = สัมพันธ์แรง 3 = ยางบางส่วน ไม่ใช่ตารางเต็ม)

House of Quality – ชั้นที่ 2: ให้คะแนนความสัมพันธ์ CR × TC (Relationship Matrix)

ในตารางความสัมพันธ์ ที่ให้คะแนน $CR-01 \times TC-01 = 9$ เพราะความแม่นยำในการจำแนกตอบสนองความต้องการ CR-01 ตรงตัวที่สุด ส่วน $CR-01 \times TC-02 = 3$ เพราะจำนวน channel มีผลต่อความแม่นยำบ้างแต่ไม่ใช่ปัจจัยหลัก — สังเกตว่าการให้คะแนนต้องมีเหตุผลรองรับทุกช่อง ไม่ใช่กรอกให้เต็มตาราง

ตอนดูหลังคาบ้าน ทีมพบข้อขัดแย้งสำคัญสองคู่: ความแม่นยำ (TC-01) กับจำนวน channel ที่น้อย (TC-02) ขัดกัน เพราะยิ่งใช้ channel น้อย ข้อมูลยิ่งน้อยลง โมเดลยิ่งแม่นยำมากขึ้น และความแม่นยำ (TC-01) กับความเร็ว (TC-03) ก็ขัดกันเช่นกัน เพราะโมเดลที่แม่นยำขึ้นมักซับซ้อนขึ้นและใช้เวลาประมวลผลนานขึ้น

การเห็นข้อขัดแย้งนี้ล่วงหน้าทำให้ทีมรู้ว่าต้องหาจุดสมดุล ไม่ใช่ไล่ตามตัวเลขสูงสุดของทุกด้านพร้อมกัน



เมื่อคำนวณน้ำหนักรวมแล้ว TC-01 (ความแม่นยำ) ได้คะแนนความสำคัญสูงสุด ทีมจึงตั้งเป้าหมายเบื้องต้นไว้ที่ $\text{macro-F1} \geq 0.80$ ส่วน TC-02 ตั้งไว้ที่ไม่เกิน 2 channel และ TC-03 ตั้งไว้ไม่เกิน 1 วินาทีต่อ epoch

เมื่อทำครบทั้งสี่ก้าวย่อยแล้ว ทีมประกอบทุกอย่างเป็น House of Quality ใบเดียว — นี่คือสิ่งที่ต้องส่งจริงในขั้นนี้ ไม่ใช่แค่พูดถึงแต่ละส่วนแยกกัน:

CR (น้ำหนัก)	TC-01 ความแม่นยำ	TC-02 จำนวน channel	TC-03 latency	TC-04 ความชัด hypnogram	TC-05 ความทนต่อ artifact
CR-01 (5)	9	3	—	—	—
CR-02 (4)	—	9	—	—	—
CR-03 (3)	—	—	9	—	—
CR-04 (3)	—	—	—	(ทีมยังไม่ให้คะแนน — ดูหมายเหตุ)	—
CR-05 (4)	—	—	—	—	9
เป้าหมาย (target)	$\text{macro-F1} \geq 0.80$	$\leq 2 \text{ channel}$	$\leq 1 \text{ วินาที/epoch}$	$\geq 80\%$ ผู้ใช้เข้าใจ*	$\leq 2\% \text{ data loss}^*$

*หมายเหตุใต้บ้าน: ตารางนี้แสดงเฉพาะคู่ที่ทีมให้เหตุผลรองรับไว้จริง (5 คู่ตามที่อธิบายไว้ข้างต้น) ส่วนช่องที่เว้นว่าง (—) หมายถึงทีมพิจารณาแล้วว่าไม่มีความสัมพันธ์ที่มีนัยสำคัญ ไม่ใช่ลืมกรอก — ในงานจริงนักศึกษาคควรพิจารณาให้คะแนนทุกคู่ที่เป็นไปได้ก่อนตัดสินใจเว้นว่าง ช่อง CR-04×TC-04 ทีมทำเครื่องหมายไว้ว่ายังไม่ได้ประเมิน เพื่อเตือนตัวเองว่าต้องกลับมาให้คะแนนก่อนส่งงานจริง (เว้นว่างแบบ “ยังไม่ทำ” กับเว้นว่างแบบ “ประเมินแล้วว่าไม่สัมพันธ์” ต้องแยกให้ชัด — ในตัวอย่างนี้จึงใจโชว์ทั้งสองแบบให้เห็นความต่าง) ส่วนบนหลังคา ทีมพบข้อขัดแย้ง 2 คู่และการเสริมกัน 1 คู่:

TC-01 $\leftrightarrow$ TC-02 = - (channel น้อยลง แม่นยำขึ้น), TC-01 $\leftrightarrow$ TC-03 = - (โมเดลแม่นยำขึ้นมักช้าลง),
TC-01 $\leftrightarrow$ TC-05 = + (โมเดลที่แม่นยำมักออกแบบให้ทนสัญญาณรบกวนไปพร้อมกัน)

จากนั้นทีมนำ TC ที่สำคัญที่สุดมาเขียนเป็น QR ที่ครบทั้ง 5 องค์ประกอบ — deliverable ของ 3–5 ตัว ทีมเลือกเขียนให้ครบทุก TC ทั้ง 5 ตัว เพื่อไม่ให้ CR ตัวไหนกลายเป็น “ไม่มีใครรับผิดชอบ” (orphan) ตอนตรวจ RTM ในเทอมถัดไป:

QR-01 (สืบย้อนได้: CR-01 $\rightarrow$ TC-01 $\rightarrow$ QR-01): “*บนชุดทดสอบ held-out แบบ subject-independent (≥ 10 คน, มี label จากผู้เชี่ยวชาญ) โมเดลจำแนก 5 ระยะการนอนได้ macro-F1 ≥ 0.80 และ Cohen’s K ≥ 0.75 โดยใช้ EEG ≤ 2 channel และ latency ≤ 1 วินาทีต่อ epoch*”

ลองแยกดูว่าประโยคนี้นี้มีครบ 5 องค์ประกอบหรือไม่: **metric** คือ macro-F1 และ K, **operator/threshold** คือ ≥ 0.80 และ ≥ 0.75 , **dataset** คือชุดข้อมูล held-out แบบ subject-independent อย่างน้อย 10 คน, และ **condition** คือใช้ไม่เกิน 2 channel และ latency ไม่เกิน 1 วินาที — ครบทุกองค์ประกอบ จึงเป็นข้อกำหนดที่ทดสอบได้จริงในเทอมถัด ๆ ไป

ทีมเขียน QR ที่เหลืออีก 4 ตัวด้วยวิธีคิดแบบเดียวกัน แต่ไม่ได้แจกแจงที่ละองค์ประกอบซ้ำอีก (ลองแยกเองเป็นแบบฝึกหัด):

QR-02 (CR-02 $\rightarrow$ TC-02 $\rightarrow$ QR-02): “เมื่อตรวจสอบสถาปัตยกรรม pipeline ทั้งระบบ (Inspection) ระบบต้องประมวลผลได้จากสัญญาณ EEG ไม่เกิน 2 channel ในทุก configuration ที่ระบุไว้ในสเปกระบบ”

QR-03 (CR-03 $\rightarrow$ TC-03 $\rightarrow$ QR-03): “บนเครื่องเป้าหมายตามสเปกขั้นต่ำที่ระบุไว้ ระบบประมวลผล 1 epoch (สัญญาณ 30 วินาที) เสร็จภายใน ≤ 1 วินาที วัดจาก 100 epoch ติดต่อกันบนชุดข้อมูลทดสอบ โดยรายงานค่า p95 latency”

QR-04 (CR-04 $\rightarrow$ TC-04 $\rightarrow$ QR-04): “ผู้ใช้ทั่วไปที่ไม่มีพื้นฐานทางการแพทย์อย่างน้อย 5 คน ตอบคำถามความเข้าใจ hypnogram ถูกต้อง $\geq 80\%$ หลังดูตัวอย่างพร้อมคำอธิบายสั้นไม่เกิน 2 นาที”

QR-05 (CR-05 $\rightarrow$ TC-05 $\rightarrow$ QR-05): “ระบบรันต่อเนื่องบนข้อมูลจำลองการนอนเต็มคืน (≥ 7 ชั่วโมง/รอบ) อย่างน้อย 10 รอบ โดยมีอัตราข้อมูลสูญหายจาก artifact ไม่เกิน 2% ของ epoch ทั้งหมด และไม่มี crash แม้แต่ครั้งเดียว”

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- House of Quality ครบใบเดียว (ตาราง 1 ใบ) พร้อมคำอธิบายสั้นได้บ้าน
- สเปก QR ครบ 5 องค์ประกอบ สำหรับ TC ที่สำคัญที่สุด 3–5 ตัว

จำนวนที่แนะนำ: CR 5–12 (ยกมาจากบทก่อน) · TC 6–15 · คู่ความขัดแย้งบนหลังคาอย่างน้อย 2 คู่ (สำคัญมากสำหรับโครงงาน AI) · QR 3–5

ข้อผิดพลาดที่พบบ่อย: TC เป็นคำลอย ๆ วัดไม่ได้ (“ระบบดี”) · มี CR ที่ไม่มี TC ตอบเลย · สัมใส่ความขัดแย้งบนหลังคา (โดยเฉพาะความแม่นยำกับความเร็ว) · QR ไม่มีชุดข้อมูลหรือเงื่อนไข ทำให้ทดสอบไม่ได้จริง

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่าน	2 ต้องแก้	1 ยังไม่ถึง
TC↔CR สืบย้อนได้ครบสองทาง (25)	ครบทุกทาง ไม่มีจุดขาด	จุดขาด ≤1 (ไม่ใช่ตัวสำคัญ)	จุดขาด 2–3	โครงความสัมพันธ์ ขาด
QR ครบ 5 องค์ประกอบ (25)	ครบทุกตัว วัดได้จริง	ขาด ≤1 องค์ ใน ≤1 QR	หลายตัวขาดองค์ ประกอบ	วัดไม่ได้/ไม่มี
TC วัดได้ + อิงมาตรฐาน (15)	วัดได้ทุกตัว จับหมวดชัด	ส่วนใหญ่วัดได้	หลายตัวเป็น feature	เป็นคำลอยล้วน
หลังคามีความขัดแย้ง ของ AI (15)	ครบ มีทางเลือกที่จะเดิน	มีหลักแต่ไม่ลึก	มีแต่ไม่แตะ AI	ไม่มี
CR ครบ + น้ำหนัก (10)	ครบทุกตัวมีเหตุผล	ครบ บางน้ำหนัก ไม่มีเหตุผล	ขาดบางตัว	ไม่มีน้ำหนัก
ตัวเลขเป้าหมายครบ (10)	ทุกตัวมี + อ้างที่มา	ส่วนใหญ่มี	บางส่วน	ไม่มี

น้ำหนักรวม 100 คะแนน: การสืบย้อนได้และความวัดได้จริง (สองเกณฑ์แรก) รวมกัน 50%

ตรวจสอบตนเองก่อนส่งงาน

- [] CR ทุกตัวมาจากบทก่อนและมีน้ำหนักครบ
- [] TC ทุกตัววัดเป็นตัวเลข/หน่วยได้ และจับหมวดมาตรฐานได้
- [] ไม่มี TC "ลอย" — ทุกตัวโยงลงหา CR อย่างน้อย 1 ตัว
- [] ไม่มี CR "ไร้คนตอบ" — ทุกตัวมี TC สัมพันธ์แรงอย่างน้อย 1 ตัว
- [] หลังคามีความขัดแย้งของ AI อย่างน้อย 2 คู่
- [] TC ทุกตัวมีตัวเลขเป้าหมาย
- [] QR สำคัญ 3–5 ตัว ครบ 5 องค์ประกอบ: metric / operator / threshold / dataset / condition
- [] เดินเส้นสืบย้อน 1 เส้นเต็มได้: CR → TC → QR

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจ "House of Quality + Quantifiable Requirements" ของโครงการวิศวกรรมคอมพิวเตอร์ ตรวจอย่างเข้มงวดตามเกณฑ์ด้านล่าง รายงานผลเป็นตาราง PASS/FAIL + เหตุผลสั้น + จุดที่ต้องแก้

ข้อมูลของฉัน:

- รายการ CR (พร้อมน้ำหนัก): [วาง]
- รายการ TC (พร้อมหมวดมาตรฐาน): [วาง]
- Relationship matrix CRxTC (9/3/1): [วาง]
- ความขัดแย้งบนหลังการระหว่าง TC: [วาง]
- ตัวเลขเป้าหมายของแต่ละ TC: [วาง]
- รายการ QR: [วาง]

ตรวจทีละข้อ:

1. CR ทุกตัวมีน้ำหนักครบไหม — ชี้ตัวที่ขาด
2. TC ตัวไหน "วัดไม่ได้" หรือเป็นแค่ชื่อ feature — ชี้ตัว
3. มี TC ตัวไหน "ลอย" ไม่โยงลงหา CR เลยไหม — ชี้ตัว
4. มี CR ตัวไหนที่ไม่มี TC สัมพันธ์แรงตอบไหม โดยเฉพาะ CR น้ำหนักสูง — ชี้ตัว
5. หลังคามีความขัดแย้งของงาน AI (เช่น ความแม่นยำ↔latency↔จำนวน channel) ไหม — ถ้าไม่มีให้เตือน
6. TC ทุกตัวมีตัวเลขเป้าหมายไหม — ชี้ตัวที่ขาด
7. QR แต่ละตัวครบ 5 องค์ประกอบไหม — ชี้องค์ที่ขาดรายตัว

สุดท้าย:

- วาด traceability chain ที่สมบูรณ์อย่างน้อย 1 เส้น: CR-?? → TC-?? → QR-?? และชี้จุดที่ขาด
 - สรุป 3 อย่างที่ควรแก้ก่อนเป็นอันดับแรก
- อย่าแต่งข้อมูลเพิ่มเอง ถ้าข้อมูลไม่พอให้บอกว่าต้องการอะไร

บทที่ 5 — จาก QFD สู่ Functional Decomposition: ออกแบบระบบ

นี่คือจุดที่ชื่อของกระบวนการทั้งหมด “QFD ไปยัง Functional Decomposition” เกิดขึ้นจริง

เมื่อมีเป้าหมายที่วัดได้ (QR/TC) จากบทก่อนแล้ว คำถามที่ตามมาคือ

“แล้วระบบต้องทำอะไรบ้างถึงจะบรรลุเป้าหมายเหล่านั้น”

บทนี้แปลงตัวเลขเป้าหมายให้กลายเป็นโครงสร้างของหน้าที่ (function)

แล้วจับหน้าที่เหล่านั้นลงในโมดูลซอฟต์แวร์จริง เพื่อให้เขียนโค้ดได้โดยทุกส่วนตอบ requirement ที่มีอยู่จริง ไม่ใช่เขียนไปเรื่อยตามความคุ้นเคย

แนวคิดเบื้องต้น

Functional Decomposition คือการแตกหน้าที่รวมของระบบออกเป็นหน้าที่ย่อยไปเรื่อย ๆ

เริ่มจากเขียนหน้าที่รวมทั้งหมดเป็นประโยคเดียว (กริยา + กรรม เช่น “ประเมินระยะการนอนจากสัญญาณ EEG”) แล้วค่อย ๆ แยกย่อยลงไปตามการไหลของข้อมูลหรือสัญญาณในระบบ

วิธีคิดแบบนี้มาจากหลักการออกแบบวิศวกรรมอย่างเป็นระบบ (systematic engineering design)

ซึ่งใช้กันแพร่หลายในหลายสาขาวิศวกรรม ไม่ใช่เฉพาะซอฟต์แวร์

กฎที่สำคัญที่สุดของขั้นนี้คือ **ทุกโหนดในแผนผังต้องเป็น “หน้าที่” ไม่ใช่ “module” หรือ “เทคโนโลยี”** —

พูดอีกแบบคือต้องเป็นคำตอบของคำถาม “ระบบทำอะไร” ไม่ใช่ “ระบบทำด้วยอะไร” ตัวอย่างเช่น

“จำแนกระยะการนอน” เป็นหน้าที่ แต่ “React frontend” หรือ “ใช้ PostgreSQL” เป็นเทคโนโลยี —

การปนกันระหว่างสองสิ่งนี้เป็นความผิดพลาดที่พบบ่อยที่สุดของขั้นนี้ เพราะมันทำให้แผนผังกลายเป็นแค่ diagram ของ tech stack แทนที่จะเป็นแผนผังความคิด

และที่สำคัญไม่แพ้กัน **ทุกหน้าที่ต้องชี้ขึ้นไปหา QR/TC ที่มันตอบสนอง** —

ถ้าแตกหน้าที่ออกมาแล้วตอบไม่ได้ว่ามันมีไว้ทำไม (ไม่มี requirement รองรับ)

หน้าที่นั้นอาจเป็นงานที่ทำเกินความจำเป็น หรือที่เรียกกันว่า scope creep

กฎนี้ช่วยกันไม่ให้ทีมหลงไปสร้างสิ่งที่ไม่มีความต้องการ และในทางกลับกัน QR ทุกตัวก็ต้องมีหน้าที่รับผิดชอบ

ไม่เช่นนั้นเป้าหมายที่ตั้งไว้จะไม่มีทางบรรลุได้จริง

เมื่อพูดถึงความต้องการด้านฟังก์ชัน (functional requirement หรือ FR) กับความต้องการด้านคุณภาพ (non-functional requirement หรือ NFR) ทั้งสองมีลักษณะต่างกันในแผนผังนี้: **FR แปลงเป็นหน้าที่ย่อยได้โดยตรง** (leaf function หนึ่งตัวมักตอบ FR หนึ่งข้อ) แต่ **NFR มักไม่ใช่หน้าที่แยก**

มันเป็นข้อจำกัดที่ครอบคลุมหลายหน้าที่พร้อมกัน เช่น “latency ไม่เกิน 1 วินาที” ไม่ใช่หน้าที่ที่แยกต่างหาก แต่เป็นเงื่อนไขที่บีบวิธีการทำงานของหน้าที่ที่เกี่ยวข้อง

การเข้าใจความต่างนี้ช่วยไม่ให้แผนผังเต็มไปด้วยหน้าที่ปลอมที่จริง ๆ แล้วเป็นแค่ข้อจำกัด

เมื่อได้แผนผังหน้าที่แล้ว ขั้นตอนต่อไปคือจับหน้าที่เหล่านั้นลงในโมดูลซอฟต์แวร์จริง (component)

หลักการสำคัญที่ใช้ในขั้นนี้มาจากแนวคิดของ David Parnas เรื่อง **information hiding** (การซ่อนข้อมูล) —

แต่ละโมดูลควรถูกออกแบบให้ซ่อน “ความลับ” หนึ่งอย่างไว้ข้างใน

โดยความลับนี้หมายถึงการตัดสินใจในการออกแบบที่มีแนวโน้มจะเปลี่ยนแปลงในอนาคต เช่น ตัวโมเดล AI ที่ใช้หรือรูปแบบของข้อมูล — ถ้าออกแบบให้โมเดลอื่นไม่ต้องรู้รายละเอียดภายในของความลับนี้เมื่อถึงเวลาต้องเปลี่ยน (เช่น เปลี่ยนโมเดล AI ตัวใหม่) จะแก้ไขแค่โมเดลเดียว ไม่กระทบส่วนอื่นของระบบนี่คือเหตุผลที่การจัดกลุ่มโมเดลไม่ควรทำตามลำดับขั้นตอนการทำงาน แต่ควรทำตามว่าอะไรมี “ความลับ” ร่วมกัน

ตัวอย่าง EEG SleepInsight

ทีมงานเริ่มจากเขียนหน้าที่รวมของระบบเป็นประโยคเดียว: “ประเมินระยะการนอนจากสัญญาณ EEG แล้วแสดงผลที่เข้าใจง่ายให้ผู้ใช้” จากนั้นแตกตามการไหลของสัญญาณ ตั้งแต่รับสัญญาณเข้ามาประมวลผลเบื้องต้น จำแนกระยะ ไปจนถึงแสดงผล — พร้อมโยงแต่ละหน้าที่กลับไปหา QR/TC ที่มันตอบสนองทุกตัว:

F	หน้าที่	ตอบสนอง
F-01	นำเข้าสัญญาณ EEG (channel น้อย)	TC-02
F-02	ประมวลผลสัญญาณเบื้องต้น (filter, ตัด epoch 30 วินาที, จัดการ artifact)	TC-05 / QR-05
F-03	สกัดคุณลักษณะ (feature extraction)	TC-01 (สนับสนุน)
F-04	จำแนกระยะการนอน 5 ระยะ	TC-01 / QR-01
↳ F-04.1	เตรียม input ให้โมเดล	ย่อยของ F-04
↳ F-04.2	รันโมเดลจำแนก	QR-01
↳ F-04.3	แปลง output เป็น label 5 ระยะ + ความมั่นใจ	QR-01
F-05	สร้าง hypnogram + สรุปคุณภาพการนอน	TC-04 / QR-04
F-06	แสดงผลให้ผู้ใช้	CR-04 / TC-04

latency และความเสถียรเป็นตัวอย่างของความต้องการด้านคุณภาพ (NFR) ที่ไม่ได้กลายเป็นหน้าที่แยก แต่ถูกวางเป็นข้อจำกัดที่บีบการทำงานของหน้าที่ที่มีอยู่แล้ว ทีมสรุปข้อจำกัดทั้งหมดไว้เป็นตารางแยกจากแผนผังหน้าที่เพื่อไม่ให้ปนกับ F ที่เป็นหน้าที่จริง

ความต้องการด้านคุณภาพ (NFR)	ผูกกับ	รายละเอียดข้อจำกัด
latency ≤ 1 วินาที/epoch (TC-03)	F-04.2 (รันโมเดลจำแนก)	บีบเวลาทำงานของขั้นตอนรันโมเดลโดยตรง ไม่ใช่หน้าที่แยก
ทำงานเสถียรตลอดคืน (TC-05)	สถาปัตยกรรมรวม (C-01 → C-02 pipeline)	ทั้ง pipeline ต้องทนสัญญาณขาด/ไม่ล่มระหว่างคืน ไม่ใช่หน้าที่เดียว รับผิดชอบ
จำนวน channel ≤ 2 (TC-02)	C-01 (Signal Ingest)	จำกัด input ที่ระบบยอมรับได้ตั้งแต่จุดรับสัญญาณที่ระบบยอมรับได้

เมื่อจับหน้าที่ลงในโมดูล ทีมจัดกลุ่มตาม “ความลับ” ที่แต่ละส่วนซ่อนไว้ แล้วสรุปเป็นตารางเดียวคู่กับแผนผังหน้าที่ด้านบน

C โมดูล	ความลับ (สิ่งที่ซ่อนไว้ข้างใน)	รับผิดชอบหน้าที่
C-01 Signal Ingest	โปรโตคอล/รูปแบบการรับสัญญาณจากอุปกรณ์ EEG แต่ละรุ่น	F-01
C-02 Preprocessing	พารามิเตอร์ filter และกฎตัดสินว่า epoch ไหนเป็น artifact	F-02, F-03
C-03 Sleep-Stage Classifier	สถาปัตยกรรมโมเดลและค่าน้ำหนักภายใน	F-04 (รวม F-04.1–F-04.3)
C-04 Result Builder	วิธีคำนวณสรุปคุณภาพการนอนจาก label รายระยะ	F-05
C-05 UI	เทคโนโลยี/เฟรมเวิร์กที่ใช้แสดงผล	F-06

ความลับของ C-03 คือสถาปัตยกรรมโมเดลและค่าน้ำหนักภายใน ซึ่งเป็นสิ่งที่คาดว่าจะเปลี่ยนบ่อยที่สุดตลอดการพัฒนา ทีมจึงออกแบบให้โมดูลอื่นรู้จัก C-03 ผ่านอินเทอร์เฟซเดียวคือ `classify(epoch) → (label, confidence)` เท่านั้น ผลคือถ้าวันหนึ่งต้องเปลี่ยนโมเดลจากสถาปัตยกรรมหนึ่งไปอีกสถาปัตยกรรมหนึ่ง จะแก้ไขแค่ภายใน C-03 โดยไม่กระทบโมดูลอื่นเลย นี่คือประโยชน์ที่จับต้องได้ของการซ่อนข้อมูล — โมดูลอื่นอย่าง C-01 หรือ C-05 ก็ใช้หลักคิดเดียวกัน แม้ความลับของมันจะเปลี่ยนไม่บ่อยเท่า C-03 ก็ตาม

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- แผนผังหน้าที่ (function tree) พร้อมการโยงแต่ละหน้าที่กลับไปหา QR/TC
- แผนผังสถาปัตยกรรม/โมดูล (C) พร้อมการจับคู่ $F \leftrightarrow C$
- รายการความต้องการด้านคุณภาพที่วางเป็นข้อจำกัด พร้อม “ความลับ” ของแต่ละโมดูล

จำนวนที่แนะนำ: หน้าที่ (F) 8–25 ตัว ลึก 2–3 ชั้น · โมดูล (C) 4–10 ตัว

ข้อผิดพลาดที่พบบ่อย: โหนดเป็นชื่อโมดูล/เทคโนโลยีแทนหน้าที่ · หน้าที่ที่ไม่มี requirement รองรับ (scope creep) · QR บางตัวไม่มีหน้าที่รับผิดชอบ · เอาความต้องการด้านคุณภาพมาเป็นหน้าที่แยก

· ไม่ระบุความลับของแต่ละโมดูล

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่าน	2 ต้องแก้	1 ยังไม่ถึง
หน้าที่↔ความต้องการ ไม่มีจุดขาด (25)	ครบสองทาง ไม่มีจุดขาด	จุดขาด ≤1 ไม่สำคัญ	จุดขาด 2-3	หลายจุดลอย
หน้าที่↔โมดูล ครบสองทาง (20)	ครบทุกทาง	จุดขาด ≤1	จุดขาด 2-3	ขาด/ไม่มี
การซ่อนข้อมูลชัดเจน (20)	ทุกโมดูลระบุความ ลับ ชัดเจน	ส่วนใหญ่ชัด	ตื่น/รู้บางส่วน	ไม่มีความลับ
โน้ตเป็นหน้าที่แท้ (15)	ทุกโน้ตเป็นหน้าที่	ส่วนใหญ่ใช่	หลายโน้ตเป็น tech	เป็น tech stack
ข้อจำกัดวางถูกที่ (10)	ทุกตัวชัดเจน	ส่วนใหญ่ถูก	บางตัวเป็นหน้าที่	ปนเป็นหน้าที่
การแตกกลีฟอดี (10)	ชัดเจน เหมาะสม	ส่วนใหญ่ดี	ลึก/ตื่นเกินบางกึ่ง	โครงมั่ว

น้ำหนักรวม 100 คะแนน: การสืบย้อนได้กลางเส้นและการซ่อนข้อมูลรวมกันเกือบ 90%
เพราะสองอย่างนี้คือแก่นของการออกแบบที่ดี

ตรวจสอบตนเองก่อนส่งงาน

- [] ทุกโน้ตในแผนผังเป็น "หน้าที่" ไม่ใช่โมดูล/เทคโนโลยี
- [] หน้าที่ทุกตัวชี้ขึ้นหา requirement ที่มันตอบสนอง (ไม่มีที่ลอย)
- [] ความต้องการทุกข้อมีหน้าที่รับผิดชอบ (ไม่มีที่ตกหล่น)
- [] ความต้องการด้านคุณภาพวางเป็นข้อจำกัด ไม่ใช่หน้าที่แยก
- [] หน้าที่ทุกตัวจับลงโมดูล และโมดูลทุกตัวมีหน้าที่รับผิดชอบ
- [] แต่ละโมดูลระบุ "ความลับ" ที่ซ่อนไว้ชัดเจน
- [] การแตกโมดูลหรือตื่นเกินไป แต่ละกึ่งทำงานเดี่ยวชัด
- [] เดินเส้นสืบย้อนได้เต็ม: requirement → หน้าที่ → โมดูล

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจ "Functional Decomposition + Architecture" ของโครงงานวิศวกรรม
ตรวจเข้มงวดตามเกณฑ์ รายงานเป็นตาราง PASS/FAIL + เหตุผล + จุดที่ต้องแก้

ข้อมูลของฉัน:

- Overall function + function tree (F nodes + ลำดับชั้น): [วาง]
- ตาราง allocation $F \rightarrow QR/TC$: [วาง]
- รายการ QR/TC: [วาง]
- Architecture/module map + $F \leftrightarrow C$: [วาง]
- ความต้องการด้านคุณภาพ + ความลับของแต่ละโมดูล: [วาง]

ตรวจที่ละข้อ:

1. โหนดไหนเป็น "module/เทคโนโลยี" แทนที่จะเป็นหน้าที่ — ชี้ตัว
2. มีหน้าที่ตัวไหน "ลอย" ไม่โยง requirement เลยไหม — ชี้ตัว
3. มี requirement ตัวไหนที่ไม่มีหน้าที่รับผิดชอบเลยไหม — ชี้ตัว
4. ความต้องการด้านคุณภาพตัวไหนถูกวางผิดเป็นหน้าที่แยก — ชี้ตัว
5. มีหน้าที่ที่ไม่ได้จับลงโมดูล หรือโมดูลที่ไม่มีหน้าที่ไหม — ชี้ orphan
6. สิ่งที่เคยคาดว่าจะเปลี่ยนบ่อย (เช่น โมเดล AI) ถูกซ่อนในโมดูลเดียวไหม — ถ้ากระจายหลายโมดูล ดีขึ้น
7. กิ่งไหนแตกลึก/ตันผิดปกติ

สุดท้าย:

- วาด traceability เต็ม 1 เส้น: requirement $\rightarrow$ หน้าที่ $\rightarrow$ โมดูล และชี้จุดขาด
 - สรุป 3 อย่างที่ต้องแก้ก่อน
- อย่าแต่งข้อมูลเพิ่มเอง

บทที่ 6 — ลงมือสร้างและทดลอง

เมื่อออกแบบเสร็จแล้ว ขั้นถัดไปคือลงมือสร้างจริง แต่คู่มือนี้ยังคงวัดกระบวนการอยู่เสมอ — การสร้างในขั้นนี้ต้องยึดติดกับสิ่งที่ออกแบบไว้ในบทก่อน และเดินเข้าหาตัวเลขเป้าหมายอย่างมีระบบ ไม่ใช่ลองผิดลองถูกแบบไม่มีทิศทาง

แนวคิดเบื้องหลัง

หลักการแรกและสำคัญที่สุดของขั้นนี้คือ **ทุกสิ่งที่สร้างต้องสืบย้อนกลับไปหาโมดูลหรือหน้าที่ที่ออกแบบไว้ได้** ถ้าระหว่างทางพบว่าต้องสร้างอะไรที่ไม่มีในแผนผังเดิม ให้ย้อนกลับไปเติมในขั้นออกแบบก่อน ไม่ใช่สร้างไปเลยแล้วค่อยหาเหตุผลย้อนหลัง เพราะนั่นจะทำให้ traceability ที่สร้างมาตลอดสามบทก่อนหน้านี้ขาดตอน

สำหรับโครงการที่มีแกน AI การทดลอง (เช่น การเทรนโมเดล ทดสอบ hyperparameter ต่าง ๆ) ควรเกิดขึ้นภายในโมดูลที่ถือความลับนั้นเท่านั้น ตามหลักการซ่อนข้อมูลที่วางไว้ในบทก่อน และควรมีบันทึกการทดลอง (experiment log) ที่จดไว้ว่าแต่ละครั้งลองอะไร ด้วยเหตุผลอะไร แล้วได้ผลอย่างไร — บันทึกนี้ไม่ใช่แค่เอกสารเพื่อส่ง แต่เป็นเครื่องมือที่ทำให้ทีมเองไม่หลงทางเมื่อทดลองหลายรอบ และช่วยให้อาจารย์ที่ปรึกษาเข้าใจกระบวนการคิดได้

อีกสิ่งที่ขาดไม่ได้สำหรับโครงการ AI คือคุณภาพของข้อมูล — เป้าหมายตัวเลขจะไร้ความหมายถ้าข้อมูลที่ใช้ไม่มีคุณภาพ ต้องประกาศให้ชัดว่าแบ่งข้อมูลฝึกสอน ตรวจสอบ และทดสอบอย่างไร และสำคัญที่สุดคือ**ต้องไม่มี data leakage** (ข้อมูลจากชุดทดสอบรั่วไปปนกับชุดฝึกสอน) เพราะถ้ามีการรั่วเกิดขึ้น ตัวเลขที่วัดได้จะดีเกินจริงและไม่มีความหมายในการใช้งานจริง

ควรทำ **model card** เอกสารสั้น ๆ ที่บันทึกว่าโมเดลนี้มีไว้ใช้กับอะไร ฝึกด้วยข้อมูลอะไร วัดผลได้เท่าไร และมีข้อจำกัดอะไรบ้าง แนวคิดนี้มาจากแนวปฏิบัติมาตรฐานในวงการ machine learning ที่ใช้สร้างความโปร่งใสให้กับโมเดลที่พัฒนาขึ้น

สุดท้าย ทุกการทดลองควร**ทำซ้ำได้ (reproducible)** — กำหนดค่า seed คงที่ ระบุเวอร์ชันของข้อมูลที่ใช้ แยกแยะชุดข้อมูลทดสอบไม่ให้เปลี่ยนระหว่างทาง และเก็บโค้ดไว้ใน version control เพื่อให้ทั้งทีมเองและอาจารย์ที่ปรึกษาสามารถกลับมาตรวจสอบผลลัพธ์ได้ในภายหลัง

ตัวอย่าง EEG SleepInsight

ทีมสร้างระบบตามโมดูลที่ออกแบบไว้ในบทก่อน (C-01 ถึง C-05) โดย C-03 คือโมดูลที่เก็บโมเดล AI และเป็นจุดที่ทีมทดลองมากที่สุด ทีมจดบันทึกการทดลองทุกครั้งเป็นตาราง ไม่ใช่แค่จำในหัวหรือเขียนสรุปตอนท้าย เพราะการทดลองด้าน AI มักมีหลายรอบจนจำเองไม่ได้ว่าลองอะไรไปแล้วบ้าง:

รอบที่	สิ่งที่เปลี่ยน	macro-F1	หมายเหตุ
1	CNN 1 channel (baseline)	0.71	โมเดลพื้นฐาน ยังไม่ปรับแต่งอะไร
2	เพิ่ม spectral feature	0.74	ดีขึ้น +0.03 จาก baseline
3	เปลี่ยนเป็น 2 channel + data augmentation	0.76	ยังห่างเป้า QR-01 (0.80) อยู่ 0.04
4 (แผนต่อไป)	ลอง ensemble หลายโมเดล + tune hyperparameter เพิ่ม	ยังไม่ทำ	คาดว่าจะปิดช่องว่างที่เหลือได้ บางส่วน

ทีมบันทึกแผนการทดลองต่อไว้อย่างชัดเจนแทนที่จะปิดบังว่ายังไม่ถึงเป้า และเขียน model card เป็นเอกสาร
แยกตามรูปแบบมาตรฐานที่ใช้กันทั่วไปในวงการ ML แทนที่จะปนไว้ในรายงานอื่น

Model Card — EEG Sleep-Stage Classifier (C-03) - Intended use: จำแนกระยะการนอน 5
ระยะจากสัญญาณ EEG ของผู้ใหญ่ระหว่างการนอน ไม่ใช่เครื่องมือวินิจฉัยโรค - **Training data:** ชุดข้อมูล
Sleep-EDF แบ่งแบบ subject-independent (คนที่อยู่ในชุดทดสอบต้องไม่ปรากฏในชุดฝึกสอนเลย เพื่อป้องกัน
data leakage) - **Data quality:** ตัดทิ้ง epoch ที่สัญญาณขาดหายเกิน 50% และควบคุมให้ทุกไฟล์มี
sampling rate เดียวกัน - **Evaluation metrics:** macro-F1 และ Cohen’s K บนชุดทดสอบที่แช่แข็งไว้ -
Known limitations: ยังไม่ได้ทดสอบกับกลุ่มผู้ป่วยที่มีความผิดปกติด้านการนอน, ยังไม่ถึงเป้า macro-F1
ที่รอบทดลองล่าสุด (0.76 จากเป้า 0.80)

และสรุปความคืบหน้าเทียบกับเป้าหมายทุกตัวเป็นตารางเดียว
เพื่อให้อาจารย์ที่ปรึกษาเห็นภาพรวมได้ในหน้าเดียวว่าตัวไหนยังต้องเร่ง ตัวไหนรอทดสอบในขั้นถัดไป:

QR	เป้าหมาย	ความคืบหน้าปัจจุบัน	สถานะ
QR-01	macro-F1 ≥ 0.80 , K ≥ 0.75	macro-F1 = 0.76 (รอบทดลองล่าสุด)	ยังไม่ถึงเป้า — มีแผนรอบ 4
QR-02	≤ 2 channel	ออกแบบสถาปัตยกรรมใช้ 2 channel แล้ว	ผ่านแล้วในระดับออกแบบ
QR-03	≤ 1 วินาที/epoch	ยังไม่วัดเวลาจริง (จะวัดหลังโมเดลนิ่งกว่านี้)	รอทดสอบ
QR-04	ผู้ใช้เข้าใจ $\geq 80\%$	ยังไม่ทดสอบกับผู้ใช้จริง	รอทดสอบ
QR-05	data loss $\leq 2\%$, 0 crash	ยังไม่รันต่อเนื่อง 8 ชั่วโมงเต็มรูปแบบ	รอทดสอบ

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- ระบบหรือโค้ดที่ทำงานได้ (สาธิตความคืบหน้า)
- บันทึกการทดลอง (experiment log)
- Model card (ถ้ามีแกน AI)
- ตารางเปรียบเทียบความคืบหน้าปัจจุบันกับเป้าหมาย QR

ข้อผิดพลาดที่พบบ่อย: สร้าง feature ที่ไม่มีในแผนผังเดิม · ไม่มีบันทึกการทดลอง (เทรนไปเรื่อยจำผลไม่ได้) · ข้อมูลรั่วระหว่างชุดฝึกสอนกับชุดทดสอบ · อ้างผลโดยไม่ประกาศชุดข้อมูล/การแบ่ง · โมเดลรั่วออกนอกโมดูลที่ควรซ่อนไว้ (ต้องแก้หลายที่เมื่อเปลี่ยนโมเดล)

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่าน	2 ต้องแก้	1 ยังไม่ถึง
สิ่งที่สร้างผูกกับแบบเสมอ (20)	ทุกส่วนสืบย้อนได้	นอกแผนเล็กน้อย	นอกแผนหลายจุด	ไม่ผูกกับแบบ
ความคืบหน้าเทียบเป้าหมายชัดเจน (20)	ตัวเลขครบ + แผนไล่ตาม	ตัวเลขส่วนใหญ่	บางส่วน	ไม่รู้ว่าทางเท่าไร
คุณภาพข้อมูล + ไม่มีรั่ว (20)	ประกาศชัด ไม่รั่ว	ส่วนใหญ่ดี	คลุมเครือ	รั่วชัดเจน
บันทึกการทดลอง + model card (20)	ครบ มีเหตุผล	มีแต่ต้น	ขาดอย่างใดอย่างหนึ่ง	ไม่มี
ทำซ้ำได้ (10)	ครบทุกองค์ประกอบ	ส่วนใหญ่มี	บางส่วน	ทำซ้ำไม่ได้
การซ่อนข้อมูลยังคงอยู่ (10)	เปลี่ยนโมเดลแต่ละที่เดียว	เกือบ	รั่วบางส่วน	กระจาย

น้ำหนักรวม 100 คะแนน: กระจายค่อนข้างสม่ำเสมอ สะท้อนว่าขั้นนี้ต้องรักษาวินัยหลายด้านพร้อมกัน

ตรวจสอบตนเองก่อนส่งงาน

- [] ทุกส่วนที่สร้างซึ่กลับโมดูล/หน้าที่ในแบบได้ (ไม่มีของนอกแผน)
- [] มีบันทึกการทดลอง: ลองอะไร ทำไม ได้ผลเท่าไร
- [] มี model card: ใช้กับอะไร ข้อมูลอะไร วัดผลได้เท่าไร ข้อจำกัดอะไร
- [] ประกาศการแบ่งข้อมูลชัด ไม่มีรั่ว + ระบุคุณภาพข้อมูล
- [] รู้ตัวเลขปัจจุบันเทียบกับเป้าหมายทุกตัว + มีแผนไล่ให้ถึง
- [] ตั้งค่าคงที่ / ข้อมูลมีเวอร์ชัน / ชุดทดสอบแช่แข็ง / โค้ดอยู่ใน version control
- [] เปลี่ยนโมเดลแต่ละแควโมดูลเดียว

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจเฟส "Implementation + ML Experimentation" ของโครงการวิศวกรรม (เน้น process + traceability)

รายงานเป็นตาราง PASS/FAIL + เหตุผล + จุดที่ต้องแก้

ข้อมูลของฉัน:

- design (โมดูล/หน้าที่จากบทก่อน): [วาง]
- รายการสิ่งที่สร้างจริง/feature: [วาง]
- บันทึกการทดลอง: [วาง]
- model card + dataset/split: [วาง]
- เป้าหมาย + ตัวเลขปัจจุบัน: [วาง]

ตรวจ:

1. feature ไหนที่สร้างแล้วไม่มีในแบบ — ชี้ (scope creep)
 2. มีความเสี่ยง data leakage ไหม — ชี้
 3. ตัวเลขปัจจุบันห่างเป้าหมายแต่ละตัวเท่าไร — ทำตาราง
 4. บันทึกการทดลอง / model card ขาดอะไร
 5. จุดที่ทำซ้ำไม่ได้
 6. โมเดลถูกซ่อนในโมดูลเดียวไหม — ถ้ารู้ ชี้จุด
- สรุป 3 อย่างที่ต้องแก้ก่อน. อย่าแต่งข้อมูลเพิ่ม

บทที่ 7 — พิสูจน์และปิดการสืบย้อน

ทุกขั้นตอนที่ผ่านมาสร้างสายโซ่ของคำสัญญาไว้อย่างเหนียวแน่น — ผู้ใช้บอกความต้องการ
ทีมแปลงเป็นตัวเลขเป้าหมาย ออกแบบระบบ และลงมือสร้าง
แต่ยังไม่มีอะไรพิสูจน์ว่าคำสัญญาเหล่านั้นถูกทำสำเร็จจริง ขั้นตอนนี้คือขั้นที่ปิดสายโซ่ให้ครบวง —
ทุกเป้าหมายที่ตั้งไว้ต้องถูกทดสอบ และผลการทดสอบต้องบันทึกไว้อย่างตรงไปตรงมา

แนวคิดเบื้องหลัง

หลักการคือเป้าหมายทุกตัวต้องจับคู่กับการพิสูจน์ที่ให้หลักฐานที่ตรวจสอบได้

วิธีการพิสูจน์มีสี่แบบมาตรฐานตามหลักวิศวกรรมระบบ: **Test** (ทดสอบภายใต้เงื่อนไขควบคุมแล้ววัดค่า),

Demonstration (สถิติให้เห็นความสามารถ), **Inspection** (ตรวจสอบชิ้นงาน/โค้ด/เอกสารโดยตรง), และ

Analysis (วิเคราะห์หรือคำนวณเมื่อการทดสอบตรงไม่สามารถทำได้จริง) —

สิ่งสำคัญคือต้องเลือกวิธีที่เหมาะสมกับสิ่งที่เราจะพิสูจน์ ตัวเลขเชิงปริมาณอย่างความแม่นยำต้องใช้ Test หรือ Analysis
ไม่ใช่แค่ Demonstration ที่โชว์ให้ดูเฉย ๆ

หลักวิศวกรรมความต้องการที่ยอมรับกันทั่วไประบุว่า ข้อกำหนดที่ดีต้อง**พิสูจน์ได้**

หมายถึงต้องมีกระบวนการที่จำกัดและคุ้มค่าที่ใครสักคนหรือเครื่องมือใดสามารถตรวจสอบได้ว่าระบบทำตามข้อกำหนดนั้นจริงหรือไม่ นี่คือเหตุผลที่คู่มือยืนยันกรานให้ทุก QR มีครบ 5 องค์ประกอบตั้งแต่บทที่ 4 —
เพราะถ้าขาดองค์ประกอบใดไป การพิสูจน์ในขั้นนี้จะทำไม่ได้เลย

เมื่อทดสอบครบแล้ว ให้รวบรวมทุกอย่างเป็น**ตารางสืบย้อนความต้องการ** (Requirements Traceability Matrix หรือ RTM) ที่บันทึกทั้งต้นทาง (ความต้องการมาจากไหน) และปลายทาง (พิสูจน์ด้วยอะไร)

ของทุกรายการ เพื่อให้แน่ใจว่าไม่มีความต้องการใดที่ไม่มีการทดสอบรองรับ

และไม่มีการทดสอบใดที่ทำไปโดยไม่มีความต้องการรองรับ

ส่วนที่สำคัญไม่แพ้การทดสอบเองคือ**ความซื่อตรงในการรายงานผล** — ถ้าเป้าหมายบางตัวไม่ผ่าน

ต้องรายงานตรงไปตรงมาพร้อมวิเคราะห์สาเหตุ ไม่ใช่ปกปิดหรือปรับตัวเลขให้ดูดี

โครงการที่รายงานผลอย่างซื่อตรงว่า “ยังไม่ถึงเป้า เพราะเหตุนี้ และมีแผนแก้ไขแบบนี้”

มีคุณค่าทางวิศวกรรมมากกว่าโครงการที่อ้างว่าผ่านทุกอย่างแต่ตรวจสอบไม่ได้จริง

ตัวอย่าง EEG SleepInsight

ทีมจับคู่ทุก QR กับการทดสอบที่เหมาะสม แล้วรวบรวมเป็นตารางการทดสอบเดียว — นี่คือการที่ต้องส่งจริง ไม่ใช่แค่เล่าผลแยกทีละตัว

QR	วิธีทดสอบ	เกณฑ์ผ่าน	ผลจริง	สถานะ
QR-01	Test (ชุดทดสอบ subject-independent ที่เข้มแข็งไว้) + Analysis (95% CI)	macro-F1 $\geq$ 0.80, $K \geq$ 0.75	macro-F1 = 0.79, $K =$ 0.73	ไม่ผ่าน (เฉียดเป้าเล็กน้อย)
QR-02	Inspection (ตรวจสอบสถาปัตยกรรม/การตั้งค่าระบบ)	$\leq$ 2 channel	ใช้ 2 channel	ผ่าน
QR-03	Test (วัดเวลาประมวลผลจริง 100 epoch)	$\leq$ 1 วินาที/epoch	0.4 วินาที (p95)	ผ่าน
QR-04	แบบสอบถามมาตรฐาน + Demonstration กับผู้ใช้จริง	$\geq$ 80% ตอบถูก	88%	ผ่าน
QR-05	Test (รันต่อเนื่อง 8 ชั่วโมง) + Analysis (อัตราสูญหาย)	data loss $\leq$ 2%, 0 crash	data loss 1.2%, 0 crash	ผ่าน

สำหรับ QR-01 ทีมเลือกรายงานผลอย่างตรงไปตรงมาพร้อมวิเคราะห์ว่าปัญหาหลักมาจากความสับสนในการจำแนกระยะ N1 ซึ่งเป็นระยะเปลี่ยนผ่านที่จำแนกยากที่สุด แทนที่จะปรับตัวเลขหรือเปลี่ยนวิธีวัดให้ผลดูดีขึ้น

เมื่อรวบรวมทุกอย่างเป็นตารางสับย่อนความต้องการฉบับสมบูรณ์ (RTM) ทีมจัดเรียงตาม CR แต่ละตัวเป็นหนึ่งแถว เดินสายโซ่เต็มตั้งแต่ต้นทางถึงปลายทาง — ตารางนี้คือหัวใจของขั้นนี้ เพราะเป็นจุดเดียวที่มองเห็นภาพรวมของทั้งกระบวนการพร้อมกันในหน้าเดียว

VoC ต้นทาง	CR	TC	QR	F	C	T	สถานะ
V4	CR-01	TC-01	QR-01	F-04	C-03	T-01	ไม่ผ่าน
V2	CR-02	TC-02	QR-02	F-01	C-01	T-02	ผ่าน
V1, V5	CR-03	TC-03	QR-03	F-04.2	C-03	T-03	ผ่าน
V3	CR-04	TC-04	QR-04	F-05, F-06	C-04, C-05	T-04	ผ่าน
V6	CR-05	TC-05	QR-05	F-02	C-02	T-05	ผ่าน

ตารางนี้ปิดครบทุกเส้นโดยไม่มีจุดขาด — ทุก CR มี T ปลายทางรองรับ และทุก T มี QR ต้นทางรองรับ (ไม่มี orphan ทั้งสองทิศทาง) แถวเดียวที่สถานะไม่ผ่านคือ CR-01 ซึ่งทีมรายงานไว้ตรงไปตรงมาแทนที่จะปิดบัง ทีมปิดท้ายด้วยรายงานสรุปผลการยอมรับสั้น ๆ หนึ่งย่อหน้า อ้างอิงกลับไปตาราง RTM ข้างต้นแทนที่จะเขียนซ้ำรายละเอียด

สรุปผลการยอมรับ (Acceptance Summary): ระบบผ่านเป้าหมาย 4 จาก 5 ตัว (QR-02 ถึง QR-05) ส่วน QR-01 (ความแม่นยำในการจำแนก) ได้ macro-F1 = 0.79 เทียบเป้า 0.80 — ต่ำกว่าเป้าหมาย 0.01 โดยสาเหตุหลักมาจากความสับสนในการจำแนกกระยะ N1 ทีมประเมินว่าระบบพร้อมสาธิตและใช้งานเพื่อคัดกรองเบื้องต้นได้ แต่ยังไม่ควรใช้ทดแทนการวินิจฉัยของผู้เชี่ยวชาญ 100% จนกว่าจะปิดช่องว่าง QR-01 ให้ถึงเป้าในรอบทดลองถัดไป

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- ตารางการทดสอบ (แต่ละเป้าหมาย → วิธีทดสอบ → เกณฑ์ผ่าน/ไม่ผ่าน → ผลจริง)
- ตารางสับย่อนความต้องการฉบับสมบูรณ์ (RTM) ตั้งแต่ต้นทางถึงปลายทาง
- รายงานสรุปผลการยอมรับ (acceptance report)

ข้อผิดพลาดที่พบบ่อย: เป้าหมายบางตัวไม่มีการทดสอบรองรับ (คำสัญญาไม่มีหลักฐาน) · มีการทดสอบที่ไม่มีเป้าหมายต้นทาง (ทดสอบมั่ว) · ใช้การสาธิตอ้างความแม่นยำเชิงตัวเลข · วัดผลบนข้อมูลชุดฝึกสอนแทนชุดทดสอบ · ปรับผลที่ไม่ผ่านให้ดูเหมือนผ่าน

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่านดี	2 ต้องแก้	1 ยังไม่ถึง
ตารางสับย่อนปิดครบไม่มีจุดขาด (30)	ครบทั้งเส้น ไม่มีจุดขาด	จุดขาด ≤ 1 ไม่สำคัญ	จุดขาด 2–3	หลายจุดไม่ปิด
ทุกเป้าหมายมีผลจริง (25)	ครบทุกตัว + ค่าจริง	ขาดเล็กน้อย	หลายตัวขาด	ไม่มีผลจริง
วัดถูกชุดข้อมูล + วิธีเหมาะสม (20)	ถูกต้องทุกตัว	ส่วนใหญ่ถูก	มีวัดผิดบางจุด	ผิดพื้นฐาน
รายงานผลตรงไปตรงมา (15)	ตรง + วิเคราะห์ดี	ตรงแต่วิเคราะห์ตื้น	คลุมเครือ	ปกปิด/แต่งผล
รายงานสรุปครบถ้วน (10)	ครบ + มีหลักฐานอ้างอิง	มีแต่ต้น	ขาดหลักฐาน	ไม่มี

น้ำหนักรวม 100 คะแนน: การปิดสายโซ่การสับย่อนหนักที่สุดที่ 55% เพราะนี่คือจุดที่พิสูจน์ว่าทุกอย่างที่ผ่านมาไม่ได้สูญเปล่า

ตรวจสอบตนเองก่อนส่งงาน

- [] เป้าหมายทุกตัวมีการทดสอบรองรับ พร้อมระบุวิธีการ
- [] ตารางสืบย้อนปิดครบทั้งเส้น ไม่มีจุดขาดที่ปลายไหน
- [] แต่ละการทดสอบมีเกณฑ์ผ่าน/ไม่ผ่าน + ผลจริงที่วัดได้
- [] วัดผลบนชุดข้อมูลที่แข็งแกร่ง ตรงตามที่ประกาศไว้ในเป้าหมาย
- [] วิธีพิสูจน์เหมาะสมกับสิ่งที่พิสูจน์ (ไม่ใช้การสาธิตอ้างตัวเลข)
- [] เป้าหมายที่ไม่ผ่าน รายงานตรงพร้อมวิเคราะห์สาเหตุ
- [] รายงานสรุปผ่าน/ไม่ผ่าน + เชื่อมโยงหลักฐาน

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจ "Verification & RTM Close-out" ของโครงการวิศวกรรม
รายงานเป็นตาราง PASS/FAIL + เหตุผล + จุดที่ต้องแก้

ข้อมูลของฉัน:

- ตารางสืบย้อนความต้องการ (RTM): [วาง]
- ตารางการทดสอบ (วิธี + เกณฑ์ + ผลจริง): [วาง]
- เป้าหมาย + ชุดข้อมูล/เงื่อนไขที่ประกาศไว้: [วาง]

ตรวจ:

1. มีจุดขาดตรงไหนในสายโซ่ทั้งเส้นไหม — ชี้
2. เป้าหมายตัวไหนไม่มีการทดสอบ หรือการทดสอบไหนไม่มีผลจริง — ชี้
3. มีการวัดตัวไหนที่ใช้ชุดข้อมูลผิดจากที่ประกาศไว้ไหม — ชี้
4. วิธีพิสูจน์ตัวไหนไม่เหมาะสม — ชี้
5. ผลที่ไม่ผ่านถูกรายงานตรงไปตรงมาไหม

สุดท้าย: เดิน traceability เต็ม 1 เส้น และชี้จุดขาด. สรุป 3 อย่างที่ต้องแก้ก่อน
อย่าแต่งข้อมูลเพิ่ม

บทที่ 8 — นำเสนอและปิดวงจรคุณค่า

การพิสูจน์ในบทก่อนตอบคำถามว่า “สร้างถูกต้องตามข้อกำหนดไว้หรือไม่” แต่ยังไม่ได้อธิบายคำถามที่สำคัญกว่านั้น: “สิ่งที่ออกแบบไว้ตั้งแต่แรกแก้ปัญหาผู้ใช้จริงหรือเปล่า” บทนี้ปิดวงจรทั้งหมดกลับไปหาจุดเริ่มต้น — เสียงของผู้ใช้ที่เก็บมาตั้งแต่บทที่ 3 — และแสดงให้เห็นว่าการเห็นว่าทุกก้าวที่ทำมามีเหตุผลและตรวจสอบได้จริง

แนวคิดเบื้องหลัง

ความแตกต่างระหว่าง **verification** (การพิสูจน์ที่ทำในบทก่อน) กับ **validation** (การตรวจสอบคุณค่าในบทนี้) คือคำถามที่ต่างกัน: verification ถามว่า “เราสร้างสิ่งนั้นถูกต้องหรือไม่” (built it right) ส่วน validation ถามว่า “เราสร้างสิ่งที่ถูกต้องหรือไม่” (built the right thing) — ระบบอาจผ่านการทดสอบทางเทคนิคทุกอย่าง แต่ถ้าไม่ตอบโจทย์ผู้ใช้จริง ก็ยังไม่ผ่าน validation ดังนั้นขั้นตอนนี้จึงต้องมีผู้ใช้หรือผู้มีส่วนได้เสียจริงมาประเมิน ไม่ใช่แค่ทีมงานประเมินกันเอง

การนำเสนอที่ดีไม่ใช่การไล่โชว์ฟีเจอร์ทีละอย่าง แต่ควรเป็นการ **เดินสายโซ่การสืบย้อน** ให้กรรมการเห็น — เลือกความต้องการสำคัญสักสองสามข้อ แล้วเดินให้เห็นตั้งแต่คำพูดของผู้ใช้จริง ผ่านทุกขั้นตอน

จนถึงหลักฐานการทดสอบ

วิธีนี้พิสูจน์ทั้งความเข้าใจของนักศึกษาเองและความสมบูรณ์ของกระบวนการทั้งหมดไปพร้อมกัน

อีกส่วนที่ขาดไม่ได้คือการ **สะท้อนกลับไปยังการตัดสินใจสำคัญ** โดยเฉพาะความขัดแย้งที่บันทึกไว้บนหลังคาของ House of Quality ตั้งแต่บทที่ 4 — ทีมเลือกทางไหน แลกกับอะไร และผลที่ได้คุ้มค่าหรือไม่ การมองย้อนกลับแบบนี้แสดงว่านักศึกษาเข้าใจว่าการออกแบบวิศวกรรมคือการตัดสินใจภายใต้ข้อจำกัด ไม่ใช่การหาคำตอบที่สมบูรณ์แบบ

และสุดท้าย ความซื่อตรงยังคงสำคัญในขั้นนี้เช่นเดียวกับบทก่อน — โครงการที่ยอมรับตรง ๆ

ว่าเป้าหมายบางตัวยังไม่ถึง พร้อมอธิบายเหตุผลและแผนต่อไป

มีคุณค่ามากกว่าโครงการที่พยายามกลบเกลื่อนหรือหลีกเลี่ยงคำถามที่ไม่สบายใจ

ตัวอย่าง EEG SleepInsight

ทีมจัดทำตารางเปรียบเทียบเป้าหมายกับผลจริงครบทุกตัว — ยกมาจากตารางการทดสอบในบทก่อนโดยตรง เพราะข้อมูลชุดเดียวกันนี้ใช้ได้ทั้งตอนพิสูจน์และตอนนำเสนอ ไม่ต้องวัดซ้ำ

QR	เป้าหมาย	ผลจริง	สถานะ
QR-01 ความแม่นยำในการจำแนก	macro-F1 $\geq$ 0.80	0.79	ไม่ถึงเป้า (ต่ำกว่า 0.01 พร้อมระบุสาเหตุ)
QR-02 จำนวน channel	$\leq$ 2 channel	2 channel	ผ่าน
QR-03 latency	$\leq$ 1 วินาที/epoch	0.4 วินาที	ผ่าน
QR-04 ความเข้าใจง่าย	$\geq$ 80% ตอบถูก	88%	ผ่าน
QR-05 ความเสถียร	data loss $\leq$ 2%, 0 crash	data loss 1.2%, 0 crash	ผ่าน

ทีมจัดให้ผู้ใช้จริง 5 คนและนักเทคนิคการนอน 1 คนทดลองใช้ระบบ ผลคือผู้ใช้ทั่วไปอ่าน hypnogram เข้าใจได้ (ตอบโจทย์ CR-04 เรื่องความเข้าใจง่าย) ส่วนนักเทคนิคการนอนให้ความเห็นว่าระบบใช้คัดกรองเบื้องต้นได้ดี แต่ยังไม่สามารถทดแทนการอ่านผลโดยผู้เชี่ยวชาญได้ทั้งหมด

ตอนนำเสนอ ทีมเลือกเดินสายโซ่จากคำพูด V4 ผ่าน CR-01, TC-01, QR-01, F-04, C-03

จนถึงหลักฐานการทดสอบ T-01 ให้กรรมการเห็นทีละขั้น พร้อมพร้อมตอบได้ทันทีเมื่อมีคำถามแทรกระหว่างทาง ทีมยังสะท้อนกลับไปว่าการตัดสินใจใช้ 2 channel แทนที่จะใช้มากกว่านั้น ซึ่งช่วยตอบโจทย์ CR-02

เรื่องความสะดวกในการใช้อุปกรณ์ แต่แลกมาด้วยความแม่นยำที่หลุดจากเป้าหมายไปเล็กน้อย

ทีมอธิบายว่านี่คือทางเลือกที่ยอมรับได้เมื่อพิจารณาจากน้ำหนักความสำคัญที่ผู้ใช้ให้ไว้ตั้งแต่บทที่ 3

และปิดท้ายด้วยข้อจำกัดที่ยังเหลืออยู่อย่างตรงไปตรงมา —

ระบบยังไม่ได้ทดสอบกับกลุ่มผู้ป่วยที่มีความผิดปกติด้านการนอน

และมีแผนจะเก็บข้อมูลเพิ่มเติมพร้อมปรับให้เหมาะกับผู้ใช้แต่ละคนมากขึ้นในอนาคต

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- รายงานการประเมินร่วมกับผู้ใช้จริง
- ตารางเปรียบเทียบเป้าหมายกับผลจริงครบทุกตัว
- สคริปต์หรือแผนการเดินสายโซ่สืบย้อนสำหรับการนำเสนอ
- การสาธิตระบบทำงานจริง พร้อมข้อจำกัดและแผนงานต่อไป

ข้อผิดพลาดที่พบบ่อย: นำเสนอแบบไล้โชว์ฟีเจอร์โดยไม่เชื่อมกลับความต้องการเดิม · ไม่มีผู้ใช้จริงมาประเมิน (มีแต่การทดสอบทางเทคนิค) · หลีกเลียงพูดถึงเป้าหมายที่ไม่ถึง · เดินสายโซ่สืบย้อนไม่ได้เมื่อถูกถาม · ไม่มีการสะท้อนกลับไปเพื่อการตัดสินใจสำคัญ

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่าน	2 ต้องแก้	1 ยังไม่ถึง
เดินสายโซ่สับย่อนได้สิ้น (25)	เดินได้ ≥ 3 เส้น ตอบคำถามเจาะได้	เดินได้ 2 เส้น มีสะดุดเล็กน้อย	บางส่วน	เดินไม่ได้
ประเมินร่วมผู้ใช้ + เชื่อมกลับความต้องการ (25)	ประเมินจริง + เชื่อมชัด	มีบางส่วน	ตื่น/เชื่อมหลวม	เป็นการโชว์พีเจอร์
เปรียบเทียบเป้าหมายกับผลจริง (20)	ครบทุกตัว ตรงไปตรงมา	ส่วนใหญ่ครบ	ขาดบางตัว	ไม่มี/เลียง
ชื่อตรง + สะท้อนการตัดสินใจ (20)	ชัดเจน + ลึก	มีแต่ตื้น	คลุมเครือ	ไม่มี/ซ่อน
สถิติทำงานจริง (10)	ครอบคลุมการใช้งานหลัก	ส่วนใหญ่ทำงาน	จำกัด	ไม่ทำงาน

น้ำหนักรวม 100 คะแนน: การสับย่อนได้และการเชื่อมโยงกลับคุณค่าของผู้ใช้รวมกัน 50%

ตรวจสอบตนเองก่อนนำเสนอ

- ☐ มีผู้ใช้จริงประเมินว่าตอบโจทย์ความต้องการเดิมไหม
- ☐ ตารางเปรียบเทียบเป้าหมายกับผลจริงครบทุกตัว
- ☐ ซ้อมเดินสายโซ่สับย่อนได้สิ้นอย่างน้อย 2-3 เส้น
- ☐ ระบุข้อจำกัดและงานต่อไปอย่างตรงไปตรงมา
- ☐ สถิติครอบคลุมการใช้งานหลัก ทำงานจริง
- ☐ สะท้อนกลับไปที่การตัดสินใจสำคัญ (เช่น ความขัดแย้งบนหลังคา) ว่าคุ้มค่าไหม
- ☐ การนำเสนอเชื่อมกลับความต้องการเดิม ไม่ใช่แค่โชว์พีเจอร์

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยชื่อนำเสนอโครงการวิศวกรรม (เน้น validation + traceability)

รายงานเป็นตาราง PASS/FAIL + เหตุผล + สิ่งที่กรรมการน่าจะถาม

ข้อมูลของฉัน:

- ตารางเป้าหมายเทียบผลจริง: [วาง]
- รายงานการประเมินกับผู้ใช้: [วาง]
- สายโซ่สืบทอดที่เตรียมเดิน: [วาง]
- ข้อจำกัด + การสะท้อนการตัดสินใจ: [วาง]

ตรวจ:

1. การนำเสนอเชื่อมกลับความต้องการเดิม หรือเป็นแค่การโชว์พีเจอร์ — ชี้จุดที่หลุด
2. สายโซ่ที่เตรียมมีจุดไหนขาด/สะดุด — ชี้
3. เป้าหมายตัวไหนไม่ถึง และถูกพูดถึงตรง ๆ ไหม
4. การสะท้อนการตัดสินใจลืกละไปไหม
5. ตั้งคำถามแบบกรรมการ 5 ข้อที่เจาะ traceability/คุณค่า พร้อมแนวทางตอบ
อย่าแต่งข้อมูลเพิ่ม

น้ำหนักคะแนนรวมทั้งกระบวนการ

ทั้ง 6 จุดตรวจในคู่มือเล่มนี้มีน้ำหนักเท่ากันเมื่อรวมเป็นคะแนนกระบวนการโดยรวม — checkpoint ละประมาณ 16.7% (คิดจาก $100\% \div 6$) โดยไม่มีจุดใดสำคัญกว่าจุดอื่นในระดับภาพรวม

เพราะแต่ละจุดตรวจล้วนเป็นห่วงโซ่ที่ขาดไม่ได้ — ถ้าจุดใดจุดหนึ่งอ่อน

สายโซ่การสืบทอดทั้งเส้นก็อ่อนตามไปด้วย

คะแนนกระบวนการนี้เป็นองค์ประกอบหนึ่งของการประเมินผลรวมทั้งวิชา

ซึ่งอาจารย์ผู้สอนเป็นผู้กำหนดสัดส่วนกับองค์ประกอบด้านผลงาน (เช่น UX/UI คุณภาพซอฟต์แวร์)

ตามดุลยพินิจของหลักสูตรต่อไป

บทสุดท้าย — รากทดแทนเมื่อไม่มีข้อมูลผู้ใช้จริง

บทนี้ใช้แทนเฉพาะ**บทที่ 3** สำหรับนักศึกษาที่ได้หัวข้อโครงงานมาแล้วและเริ่มลงมือทำไปบ้าง จนไม่มีโอกาสย้อนกลับไปเก็บความต้องการผู้ใช้ใหม่ตั้งแต่ต้น **บทที่ 4** เป็นต้นไปยังคงใช้เหมือนเดิมทุกประการ ไม่มีการเปลี่ยนแปลง

ทำไมต้องมีบทนี้

ความต้องการของผู้ใช้ (Voice of Customer) คือรากของการสืบทอดทั้งเส้นตามที่อธิบายไว้ในบทที่ 1 หากไม่มีข้อมูลผู้ใช้จริง สายโซ่ทั้งหมดจะไม่มีจุดเริ่มต้นให้ยึด แต่นั่นไม่ได้แปลว่าให้ข้ามขั้นตอนนี้ไปเลย ๆ — ต้องสร้าง “**รากทดแทนที่เชื่อถือได้**” แทน คือรายการความต้องการที่สันนิษฐานขึ้น (assumed) แต่ทุกข้อมีหลักฐานรองรับและติดป้ายชัดเจนว่าเป็นข้อสันนิษฐาน ไม่ใช่สร้างทำเป็นว่าเก็บข้อมูลจริงมา

แนวคิดเบื้องหลัง

ความต้องการที่สันนิษฐานขึ้นแตกต่างจากการเดาเดียงตรงที่**ต้องมีหลักฐานรองรับทุกข้อ** หลักฐานเหล่านี้อาจมาจากพีเจอร์ทที่ทีมสร้างไปแล้วก่อนหน้านี้ (สะท้อนว่าทีมเชื่อว่าผู้ใช้ต้องการสิ่งนั้น) จากการพูดคุยแบบไม่เป็นทางการกับคนใกล้ชิดที่อาจเป็นตัวแทนผู้ใช้ จากผลิตภัณฑ์ใกล้เคียงในตลาดที่มีพีเจอร์ทคล้ายกัน หรือจากงานวิจัยที่เกี่ยวข้อง — ไม่ว่าจะมาจากแหล่งไหน ต้องระบุให้ชัดว่ามาจากไหน เพื่อให้ตรวจสอบได้ว่าความต้องการนั้นมีที่มาจริง ไม่ใช่คิดขึ้นมาลอย ๆ

สิ่งสำคัญไม่แพ้กันคือ**ต้องแยกให้ชัดว่าอะไรคือข้อเท็จจริง อะไรคือข้อสันนิษฐาน** —

ทุกความต้องการในกลุ่มนี้ต้องติดป้าย “**สันนิษฐาน**” ไว้อย่างเปิดเผย

เพื่อไม่ให้เข้าใจผิดว่าเป็นข้อมูลที่เก็บมาจากผู้ใช้จริง และควร**ทำทะเบียนข้อสันนิษฐาน**

บันทึกไว้ว่าแต่ละข้อมีความเสี่ยงแค่ไหนหากสันนิษฐานผิด —

ข้อสันนิษฐานที่มีความเสี่ยงสูงและกระทบความต้องการสำคัญ

ควรพิจารณาตรวจสอบกับผู้ใช้ตัวแทนสักคนสองคนหากมีโอกาส แม้จะไม่ใช้ข้อบังคับก็ตาม

กับดักที่ต้องระวังที่สุดของแทร็กนี้คือการ “**หาเหตุผลย้อนหลัง**” —

คือการปรับความต้องการให้พอดีกับสิ่งที่สร้างไว้แล้วเป๊ะจนน่าสงสัย แทนที่จะยอมรับตรง ๆ

ว่ามีช่องว่างระหว่างสิ่งที่สร้างไว้กับความต้องการที่ตั้งไว้ ถ้าพบว่าพีเจอร์ทที่สร้างแล้วไม่มีความต้องการรองรับ

หรือความต้องการบางข้อยังไม่มีอะไรตอบ ให้บันทึกช่องว่างนั้นไว้ตรง ๆ แล้วอธิบายว่าจะแก้ไขอย่างไร

แทนที่จะพยายามกลบเกลื่อน

เดินทางตัวอย่าง: Intelligent Travel Planning AI Chatbot

โครงงานนี้เป็นแชทบอวางแผนการเดินทางด้วย AI ที่ทีมได้หัวข้อและเริ่มสร้างไปบ้างแล้ว

ทีมตั้งกรอบปัญหาโดยระบุผู้ใช้เป้าหมายที่สันนิษฐานไว้ว่าเป็นนักท่องเที่ยวอิสระที่อยากวางแผนทริปให้เร็วโดยไม่ต้องเปิดหลายเว็บพร้อมกัน จากนั้นสร้างรายการความต้องการที่สันนิษฐานขึ้น

พร้อมติดป้ายและระบุหลักฐานรองรับทุกข้อ:

ความต้องการที่สัณนิษฐาน	น้ำหนัก	หลักฐานรองรับ
[สัณนิษฐาน] ได้แผนเที่ยวที่ตรงงบและวัน	5	ผลิตภัณฑ์ใกล้เคียงในตลาดมีฟีเจอร์นี้, ฟีเจอร์ที่ทีมสร้างไปแล้ว
[สัณนิษฐาน] คุยเป็นภาษาไทยธรรมชาติ	4	พูดคุยกับผู้ใช้ตัวแทน 2 คนที่อยากพิมพ์ภาษาไทย, งานวิจัยด้าน NLP ภาษาไทย
[สัณนิษฐาน] แนะนำที่ตรงความชอบ	4	ผลิตภัณฑ์ใกล้เคียงในตลาด, ข้อเสนอแนะของทีม
[สัณนิษฐาน] ตอบไว ไม่รอนาน	3	แนวปฏิบัติมาตรฐานของแชทบอตทั่วไป

ทีมยังทำทะเบียนข้อสันนิษฐานควบคู่ไปด้วย บันทึกไว้ว่าข้อสันนิษฐานเรื่อง “ผู้ใช้อยากพิมพ์ข้อความภาษาไทยยาว ๆ” มีความเสี่ยงระดับกลาง เพราะถ้าผิดจะต้องเพิ่มปุ่มตัวเลือกสำเร็จรูปแทน ส่วนข้อสันนิษฐานเรื่อง “งบบประมาณเป็นเกณฑ์หลักในการตัดสินใจ” มีความเสี่ยงสูง เพราะกระทบความต้องการที่มีน้ำหนักสูงสุด ทีมจึงระบุไว้ว่าควรตรวจสอบข้อนี้กับผู้ใช้ตัวแทนเพิ่มเติมหากมีโอกาส

หลังจากได้รายการความต้องการที่สัณนิษฐานขึ้นพร้อมน้ำหนักแล้ว ทีมนำมันเข้า House of Quality เหมือนกับทีมงาน EEG SleepInsight ทุกประการในบทที่ 4 เป็นต้นไป ส่วนฟีเจอร์ที่สร้างไว้ก่อนหน้านี้ที่ไม่มีความต้องการรองรับ ทีมบันทึกไว้เป็นช่องว่างที่ต้องอธิบายหรือแก้ไข แทนที่จะพยายามหาเหตุผลมารองรับย้อนหลัง

สิ่งที่ต้องส่งเมื่อจบขั้นนี้

- Problem statement + กลุ่มผู้ใช้ที่สัณนิษฐาน
- ตารางความต้องการที่สัณนิษฐาน พร้อมป้าย หลักฐาน และน้ำหนัก
- ทะเบียนข้อสันนิษฐาน พร้อมระดับความเสี่ยง

จำนวนที่แนะนำ: ความต้องการที่สัณนิษฐาน 5–12 ข้อ · หลักฐานอย่างน้อย 1 อย่างต่อข้อ · ทะเบียนข้อสันนิษฐานครอบคลุมข้อที่มีความเสี่ยงสูงทุกข้อ

ข้อผิดพลาดที่พบบ่อย:

เขียนความต้องการที่สัณนิษฐานแบบเดาส่งเดชไม่มีหลักฐาน · เขียนราวกับว่าเป็นผลสำรวจผู้ใช้จริงทั้งที่ไม่ได้ทำ · ไม่มีทะเบียนข้อสันนิษฐาน/ไม่ประเมินความเสี่ยง · ปรับความต้องการให้เข้ากับสิ่งที่สร้างไว้แล้วเปะจนน่าสงสัย

เกณฑ์การให้คะแนน

เกณฑ์ (น้ำหนัก)	4 ดีเยี่ยม	3 ผ่านดี	2 ต้องแก้	1 ยังไม่ถึง
ทุกข้อมีหลักฐานรองรับ (25)	ชัดเจน ตรวจสอบได้	เดาเลย ≤ 1 ข้อ	เดาเลย 2-3 ข้อ	ส่วนใหญ่ไม่มีหลักฐาน
ตีป้ายสันนิษฐานตรงไปตรงมา (25)	แยกชัดทุกข้อ ไม่พราง	ส่วนใหญ่ชัด	ไม่ครบ/คลุมเครือ	พรางเป็นข้อมูลจริง
ทะเบียนข้อสันนิษฐาน + ความเสี่ยง (20)	ครบ + ประเมินดี	มีแต่ต้น	ไม่ครบ	ไม่มี
เป็นความต้องการ + น้ำหนักมีที่มา (20)	ล้วนเป็นความต้องการ	ส่วนใหญ่ดี	ปนวิธีแก้/น้ำหนักลอย	เป็น feature list
Framing ชัดเจน (10)	ชัด สมเหตุผล	ครบแต่ต้น	คลุมเครือ	ไม่ชัด

น้ำหนักรวม 100 คะแนน: หลักฐานและความชัดเจนในการตีป้ายรวมกัน 50% —

เพราะนี่คือหัวใจของการทดแทนที่น่าเชื่อถือ

(การตรวจสอบกับผู้ใช้ตัวแทนสำหรับข้อสันนิษฐานความเสี่ยงสูงเป็นคะแนนเสริม ไม่หักหากไม่ทำ)

ตรวจสอบตนเองก่อนส่งงาน

- [] มี problem statement + กลุ่มผู้ใช้ที่สันนิษฐานอย่างสมเหตุผล
- [] ความต้องการเขียนเป็น "ความต้องการ" ไม่ใช่วิธีแก้
- [] ทุกข้อมีหลักฐานรองรับอย่างน้อย 1 อย่าง ไม่มีเดาเลย
- [] ทุกข้อตีป้าย "สันนิษฐาน" ชัด ไม่พรางว่าเป็นข้อมูลจริง
- [] ทุกข้อมีน้ำหนัก + บอกที่มาของน้ำหนัก
- [] มีทะเบียนข้อสันนิษฐาน + ประเมินความเสี่ยง
- [] ไม่ได้ปรับความต้องการให้เข้ากับของที่สร้างไว้แล้ว (ถ้ามีช่องว่าง บันทึกตรง ๆ)

หรือวางข้อมูลของตนเองใน prompt นี้ให้ผู้ช่วย AI ช่วยตรวจ:

คุณคือผู้ช่วยตรวจ "รากทดแทนของความต้องการ" (สำหรับนักศึกษาที่ไม่มีข้อมูลผู้ใช้จริง)
จุดสำคัญคือความชัดเจน: แยกข้อเท็จจริงกับข้อสันนิษฐาน และกันการหาเหตุผลย้อนหลัง
รายงานเป็นตาราง PASS/FAIL + เหตุผล + จุดที่ต้องแก้

ข้อมูลของฉัน:

- Problem statement + กลุ่มผู้ใช้ที่สันนิษฐาน: [วาง]
- ตารางความต้องการ (+ ป้าย + หลักฐาน + น้ำหนัก): [วาง]
- ทะเบียนข้อสันนิษฐาน (+ ความเสี่ยง): [วาง]
- (ถ้ามี) รายการฟีเจอร์ที่สร้างไปแล้ว: [วาง]

ตรวจ:

1. ข้อไหน "เคาโลย" ไม่มีหลักฐานรองรับ — ชี้
 2. มีข้อไหนเขียนเหมือนเก็บข้อมูลจริงมาแล้วทั้งที่เป็นข้อสันนิษฐาน — ชี้
 3. ข้อไหนดู "ปรับให้เข้ากับฟีเจอร์ที่สร้างไว้เป๊ะ" น่าสงสัยว่าหาเหตุผลย้อนหลัง — ชี้
 4. ข้อสันนิษฐานความเสี่ยงสูงตัวไหนยังไม่มีในทะเบียนหรือไม่ได้ประเมินผลกระทบ
 5. มีฟีเจอร์ที่สร้างแล้วแต่ไม่มีความต้องการรองรับไหม
- สรุป 3 อย่างที่ต้องแก้ก่อน. อย่าแต่งข้อมูลเพิ่ม

หลังบทนี้ให้อ่านต่อจากบทที่ 4 เป็นต้นไปตามปกติ

ภาคผนวก — ตัวอย่างเพิ่มเติมจากโครงงานรูปแบบอื่น

ตัวอย่าง EEG SleepInsight ที่เดินตลอดเล่มเป็นโครงงานที่มีแกน AI/machine learning เป็นหลัก เพื่อให้เห็นว่าวิธีคิดเดียวกันนี้ปรับใช้ได้กับโครงงานรูปแบบอื่นด้วย

ภาคผนวกนี้สรุปสายโซ่การสืบย้อนแบบย่อจากอีกสองโครงงาน — ซอฟต์แวร์ล้วนที่ไม่มีแกน AI และระบบ IoT ที่มีฮาร์ดแวร์เป็นส่วนประกอบ แสดงเฉพาะจุดสำคัญของสายโซ่

ไม่ได้เดินซ้ำทุกขั้นตอนแบบละเอียดเหมือนตัวอย่างหลัก

ตัวอย่างที่ 1: API Gateway for Large Language Model Services (ซอฟต์แวร์ล้วน)

โครงงานนี้สร้างระบบตัวกลางที่รับคำขอจากผู้ใช้แล้วส่งต่อไปยังผู้ให้บริการโมเดลภาษาขนาดใหญ่หลายเจ้า พร้อมจัดการเรื่องโควตา ความปลอดภัย และค่าใช้จ่าย — ไม่มีแกน machine learning ของตัวเอง แต่เน้นความถูกต้องด้านสถาปัตยกรรมซอฟต์แวร์และความน่าเชื่อถือของระบบ

ชั้น	รายละเอียด
ความต้องการของผู้ใช้	นักพัฒนาต้องการสลับผู้ให้บริการโมเดลได้โดยไม่ต้องแก้ไขโค้ดแอปพลิเคชัน และต้องการควบคุมค่าใช้จ่ายไม่ให้เกินงบที่ตั้งไว้
คุณลักษณะทางเทคนิค	ความสามารถในการสลับผู้ให้บริการ (interoperability) และความแม่นยำในการติดตามค่าใช้จ่ายแบบเรียลไทม์ (accuracy of usage tracking)
เป้าหมายที่วัดได้	สลับผู้ให้บริการโดยไม่มี downtime เกิน 5 วินาที บนสภาพแวดล้อมทดสอบที่จำลอง traffic จริง; ติดตามค่าใช้จ่ายคลาดเคลื่อนไม่เกิน 1% เทียบกับใบแจ้งหนี้จริงของผู้ให้บริการ
หน้าที่ของระบบ	กำหนดเส้นทางคำขอไปยังผู้ให้บริการที่เหมาะสม (ซ่อนรายละเอียด API ที่แตกต่างกันของแต่ละเจ้าไว้หลัง interface เดียว), นับและบันทึกการใช้งานแบบเรียลไทม์
โมดูล	Router (ความลับ: การแปลงรูปแบบคำขอ/คำตอบระหว่างผู้ให้บริการแต่ละเจ้า), Usage Tracker (ความลับ: วิธีคำนวณค่าใช้จ่ายของแต่ละผู้ให้บริการซึ่งมีสูตรต่างกัน)
การพิสูจน์	ทดสอบสลับผู้ให้บริการภายใต้ traffic จำลอง วัดเวลา downtime; เปรียบเทียบตัวเลขที่ระบบติดตามกับใบแจ้งหนี้จริงหลังใช้งาน 1 เดือน

จุดที่น่าสังเกตคือแม้ไม่มีแกน AI แต่หลักการเดียวกันยังใช้ได้ครบ — โดยเฉพาะการซ่อนความแตกต่างของ API แต่ละผู้ให้บริการไว้หลัง interface เดียวในโมดูล Router ตรงตามหลักการซ่อนข้อมูลที่อธิบายไว้ในบทที่ 5 ทุกประการ

ตัวอย่างที่ 2: AIoT-based LPG Monitoring System (IoT)

โครงการนี้ติดตั้งเซนเซอร์วัดปริมาณแก๊ส LPG ในถังของร้านซักรีดแบบหยอดเหรียญ
แล้วใช้โมเดลทำนายว่าแก๊สจะหมดเมื่อไร เพื่อแจ้งเตือนให้เปลี่ยนถังล่วงหน้า — มีทั้งองค์ประกอบด้านฮาร์ดแวร์
(เซนเซอร์ อุปกรณ์ IoT) และด้าน AI (โมเดลทำนาย) ผสมกัน

ชั้น	รายละเอียด
ความต้องการของผู้ใช้	เจ้าของร้านซักรีดไม่ต้องการให้แก๊สหมดกลางคันขณะลูกค้าใช้งาน และต้องการรู้ล่วงหน้าว่าควรสั่งแก๊สใหม่เมื่อไรเพื่อไม่ให้เสียเวลา ขนส่งฉุกเฉิน
คุณลักษณะทางเทคนิค	ความแม่นยำในการทำนายเวลาที่แก๊สจะหมด (prediction accuracy) และความทนทานของเซนเซอร์ต่อสภาพแวดล้อม (reliability)
เป้าหมายที่วัดได้	ทำนายเวลาที่แก๊สจะหมดคลาดเคลื่อนไม่เกิน 6 ชั่วโมง เทียบกับเวลาจริงที่วัดได้จากข้อมูลย้อนหลัง 30 วัน; เซนเซอร์ส่งข้อมูลสำเร็จไม่ต่ำกว่า 99% ของรอบการอ่านค่าทั้งหมดในสภาพแวดล้อมร้านซักรีดจริง
หน้าที่ของระบบ	อ่านค่าน้ำหนักหรือแรงดันจากเซนเซอร์เป็นระยะ, ทำนายเวลาที่แก๊สจะหมดจากแนวโน้มการใช้งาน, ส่งการแจ้งเตือนเมื่อใกล้ถึงเกณฑ์
โมดูล	Sensor Reader (ความลับ: โปรโตคอลการสื่อสารกับฮาร์ดแวร์เซนเซอร์เฉพาะรุ่น), Depletion Predictor (ความลับ: โมเดลและวิธีคำนวณแนวโน้มการใช้งาน), Alert Dispatcher
การพิสูจน์	เปรียบเทียบเวลาที่ทำนายกับเวลาจริงที่แก๊สหมดจากข้อมูลย้อน หลัง; ทดสอบอัตราความสำเร็จของการส่งข้อมูลเซนเซอร์ในสภาพแวดล้อมจริงต่อเนื่อง 2 สัปดาห์

จุดที่น่าสังเกตในตัวอย่างนี้คือ ความไม่แน่นอนของฮาร์ดแวร์ (สัญญาณเซนเซอร์ขาดหาย
สภาพแวดล้อมไม่เอื้ออำนวย) กลายเป็นคุณลักษณะทางเทคนิคที่ต้องวัดและตั้งเป้าอย่างชัดเจน
เช่นเดียวกับความแม่นยำของโมเดล —
แสดงว่ากรอบความคิดเดียวกันนี้ครอบคลุมทั้งความไม่แน่นอนด้านซอฟต์แวร์และด้านฮาร์ดแวร์ได้พร้อมกัน